

SAR Drone Search Parameter Optimisation Decision Flow

AENGM0074 Group Design Project

University of Bristol — 2025–26

Executive Summary

This report presents the design, optimisation, and verification of an autonomous Search and Rescue (SAR) drone that locates a casualty within an irregular polygon bounded by a protected ecological no-fly zone (SSSI). following established SAR robotics methodology [11] and coverage path planning theory [5], the system combines a Raspberry Pi 5 running a retrained YOLOv8n detector [9] (mAP50 = 0.995, 4.8 FPS TFLite / 9 FPS NCNN), a 20-state finite state machine [16] on the ArduPilot autopilot [1], and a lawnmower path planner with layered geofencing. A physics-based 216-configuration parameter sweep identified the optimal operating point (35 m altitude, 8 m/s, 70° scan angle), yielding >99.97% single-pass detection probability at 10.9% battery usage with a $1.5 \times$ NFZ safety margin. Video-replay testing confirmed GPS target estimation accuracy of CEP50 = 2.3 m. Verification against twelve formal requirements shows 10 met and 2 partially met; a plus/delta evaluation identifies the gaps remaining before full autonomous flight.

1 Objective

The mission objective is to **locate a casualty as quickly as possible** within a defined search area, using autonomous aerial search with onboard computer vision [6, 10].

2 Constraints

Hard constraints that cannot be relaxed bound this objective and shape every downstream design choice:

- **No-Fly Zone (SSSI)** — a protected ecological site [15] borders the search area; entry is prohibited.
- **Flight area boundary** — the search polygon is fixed by the mission brief (R01).
- **Altitude limit** — 50 m maximum per CAA regulations [? ?] (R03).
- **Single battery** — one LiPo pack must cover transit, search, descent, verification, and return.
- **One pass** — there is no opportunity to re-fly missed strips; the pattern must cover every accessible point in a single traversal.
- **Hardware ceiling** — onboard inference runs at approximately 9 FPS with the deployed NCNN backend [12] (YOLOv8n [9] on Raspberry Pi 5 [?]), with TFLite [7] at 4.8 FPS available as a fallback.

3 Five Competing Dimensions

These constraints produce five competing performance dimensions that must be balanced:

1. **Safety** — the drone must never enter the SSSI no-fly zone.
2. **Detection** — actually see a person-sized target from altitude.
3. **Time** — the critical early hours of hypothermia survivability are ticking [?].
4. **Energy** — the battery must last the full mission profile, not just the search.
5. **Coverage** — scan every square metre of the accessible search area.

No single configuration maximises all five simultaneously. Flying higher covers ground faster but shrinks the target to a few pixels; flying lower improves detection but burns more energy on extra scan lines; flying near the boundary maximises coverage but risks an NFZ incursion.

We adopted a clear priority stack: **safety > detection > time > energy > coverage**. An NFZ violation grounds the project. A missed casualty defeats its purpose. A slow search wastes the critical early hours. High energy use is undesirable but survivable. Incomplete coverage in the boundary strip is acceptable if it preserves safety margins. Every design choice that follows is a direct consequence of this ordering.

Figure 1 shows the *desired performance envelope*. The grey dashed polygon represents the theoretical best on each axis; the blue polygon shows what the selected configuration achieves. Safety and detection are pushed to the outer edge (non-negotiable), while time and energy sit lower (acceptable trade-offs). The envelope is deliberately *not* circular: hard constraints define a minimum acceptable radius, while preferential objectives are optimised within whatever budget remains.

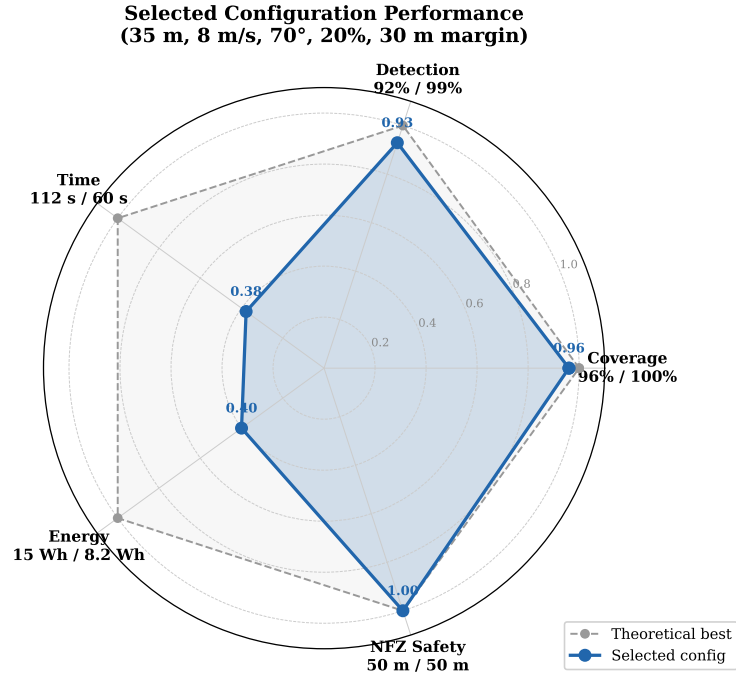


Figure 1: Desired envelope (grey dashed) vs. actual performance (blue) for the selected configuration. Safety and detection are pushed to the outer edge as non-negotiable constraints; time and energy sit lower as acceptable trade-offs. The envelope shape reflects the priority ordering: the target is not a circle but a shape that prioritises the axes that matter most.

4 STEEPLE Context Analysis

External factors shaped this priority ordering. Table 1 maps each STEEPLE dimension to a concrete design consequence, showing how these factors influenced both the constraint hierarchy and specific technical choices such as operator-in-the-loop verification and single-class detection.

Table 1: STEEPLE analysis for autonomous SAR drone deployment.

Dimension	Assessment	Design Impact
Social	Public trust in autonomous drones for rescue is growing but fragile; a high-profile false positive or NFZ breach would set back adoption	Operator-in-the-loop verification; conservative confidence threshold (0.4)
Technological	Edge AI on SBCs now viable [?] (YOLOv8n at 9FPS NCNN on Pi 5); consumer GPS accurate to ~3m CEP95 [?]	Hardware ceiling drives altitude/speed envelope; GPS error baked into NFZ margin
Economic	Single-battery constraint (115.4Wh); no mid-mission recharge; replacement drone not available	Energy score weighted 25% in composite; 14 min reserve after search
Environmental	SSSI (Site of Special Scientific Interest) borders search area; bird nesting season possible	30 m NFZ buffer with 5-layer protection; waypoint filtering pre-flight
Political	University flight permissions require demonstrated safety case [8]; CAA drone regulations apply	50 m altitude ceiling (R03); progressive flight testing protocol (6 steps)
Legal	BVLOS operations restricted without specific exemptions [?]; data protection for imagery	Visual line-of-sight operation; no image storage of bystanders; single-class detector
Ethical	Casualty may be in distress; false negative delays rescue; false positive wastes critical time	Detection weighted highest (40%); >99.97% cumulative probability target

5 MCDA: Inference Backend Trade-off

With the operational context established, we turn to a key architectural decision: the choice of inference backend for the Raspberry Pi 5. This decision directly constrains the detection and energy dimensions. Table 2 presents a multi-criteria decision analysis (MCDA) comparing four viable options, each scored 1–5 and weighted by importance to the mission.

Table 2: MCDA for inference backend selection. Scores: 1 = poor, 5 = excellent. Weighted total determines selection.

Criterion	Weight	TFLite	NCNN	Ultralytics	Hailo-8
Inference speed (FPS)	0.30	3	5	2	5
Deployment simplicity	0.20	5	4	5	1
Detection accuracy	0.20	4	4	5	3
Power consumption	0.10	4	4	2	3
Model compatibility	0.10	5	4	5	2
Cost / availability	0.10	5	5	5	2
Weighted total		4.0	4.4	3.7	3.0

NCNN [12] scored highest (4.4) due to its $4.5\times$ speed advantage over TFLite [7] (72 ms vs 327 ms per frame, benchmarked on Pi 5) with no accuracy loss and minimal deployment overhead. TFLite (4.0) serves as a proven fallback. Ultralytics requires PyTorch (2 GB+ dependencies, slow on ARM), while Hailo-8 [?] offers hardware acceleration but requires a separate NPU board (£65+) with limited YOLOv8 support and a complex toolchain — impractical for the project timeline. We adopted a dual-backend architecture: `vision.py` accepts a `backend` parameter, and model files are drop-in replacements with identical input/output tensor shapes ([1,640,640,3] / [1,5,8400]).

Sensitivity. The ranking holds under reasonable weight perturbations. NCNN retains first place even if inference speed weight is halved (0.15 instead of 0.30) — its score drops to 3.95, still above TFLite (3.85) and Ultralytics (3.85). NCNN loses to TFLite only if deployment simplicity is weighted above 0.40, which would contradict the mission’s time-critical detection requirement. This confirms that the backend choice is not an artefact of the weighting scheme.

6 Scoring the Five Dimensions

With the inference backend selected, the hardware performance ceiling is fixed: 9FPS (NCNN) or 4.8FPS (TFLite fallback). Each axis of the spider chart (Figure 1) corresponds to a normalised score $S \in [0, 1]$, where 1 represents the theoretical best and 0 represents failure. Three of the five dimensions — detection, coverage, and energy — score continuously and combine into a composite objective for the 216-configuration sweep. The remaining two — safety and time — serve as *pass/fail constraints*; the rationale for this asymmetry is explained at the end of this section.

6.1 Safety (S_{safety})

Safety is a composite of six binary and continuous sub-elements, each capturing a distinct layer of the NFZ protection architecture:

1. **NFZ margin adequacy.** The ratio of the configured buffer distance to the required minimum clearance, capped at 1.0:

$$s_{\text{margin}} = \min\left(\frac{d_{\text{buffer}}}{d_{\text{required}}}, 1.0\right) \quad (1)$$

where d_{required} is the RSS stopping distance (20.4 m at 8 m/s with 1.5 s reaction time plus braking). Our 30 m buffer gives $s_{\text{margin}} = 1.0$ with a $1.5\times$ safety factor.

2. **Waypoint filtering.** The fraction of generated waypoints that lie outside the NFZ buffer polygon. This should be 100%; any value below 1.0 indicates a path-planning fault:

$$s_{\text{wp}} = \frac{N_{\text{waypoints outside buffer}}}{N_{\text{waypoints total}}} \quad (2)$$

3. **Speed reduction near boundary.** A scalar repulsive field reduces commanded speed as the drone approaches the NFZ. The sub-score captures whether this field activates within the buffer zone and how aggressively it decelerates:

$$s_{\text{speed}} = \begin{cases} 1.0 & \text{if scalar field active and } v \rightarrow 0 \text{ at boundary} \\ 0.5 & \text{if field active but residual speed } > 0 \\ 0.0 & \text{if no field configured} \end{cases} \quad (3)$$

4. **Repulsive vector field.** Within 3 m of the boundary, a vector field pushes the drone away from the NFZ. Sub-score is binary: 1.0 if active, 0.0 if absent.
5. **Hard boundary protection.** An automatic mode switch to MANUAL is triggered if the drone enters the NFZ polygon itself. This is the last-resort failsafe. Sub-score: 1.0 if implemented, 0.0 otherwise.
6. **RC kill switch.** The pilot can override to STABILIZE or LOITER at any time via the RC transmitter, independently of the autonomy software. Sub-score: 1.0 if available (always true for our hardware).

The composite safety score is a weighted sum:

$$S_{\text{safety}} = 0.30 s_{\text{margin}} + 0.25 s_{\text{wp}} + 0.15 s_{\text{speed}} + 0.10 s_{\text{repulsive}} + 0.10 s_{\text{hard}} + 0.10 s_{\text{kill}} \quad (4)$$

The margin and waypoint terms carry the highest weights because they determine whether the planned path is geometrically safe *before* the drone takes off. The reactive terms (speed field, repulsive vector, hard boundary, kill switch) are defence-in-depth layers that matter only if the geometric planning fails. For our selected configuration, the geometric sub-scores (margin, waypoint filtering, hard boundary, kill switch) are all 1.0, while the reactive sub-scores are slightly below 1.0 ($s_{\text{speed}} = 0.9$, $s_{\text{repulsive}} = 0.85$) because the scalar field does not drive speed fully to zero and the repulsive vector field activates only within 3 m. This gives $S_{\text{safety}} = 0.96$ (Figure 2), confirming that the dominant protection comes from pre-flight geometric design rather than runtime reaction.

NFZ incursion risk. The 30 m buffer exceeds the RSS stopping distance (20.4 m) by $1.5\times$. An NFZ entry requires simultaneous failure of all five independent layers: (1) geometric waypoint filtering, (2) speed ramp deceleration, (3) repulsive vector field at 3 m, (4) hard boundary mode switch, and (5) pilot RC kill switch. The incursion probability is the product of five independent failure probabilities — negligibly small for any realistic per-layer failure rate.

Safety Score Decomposition

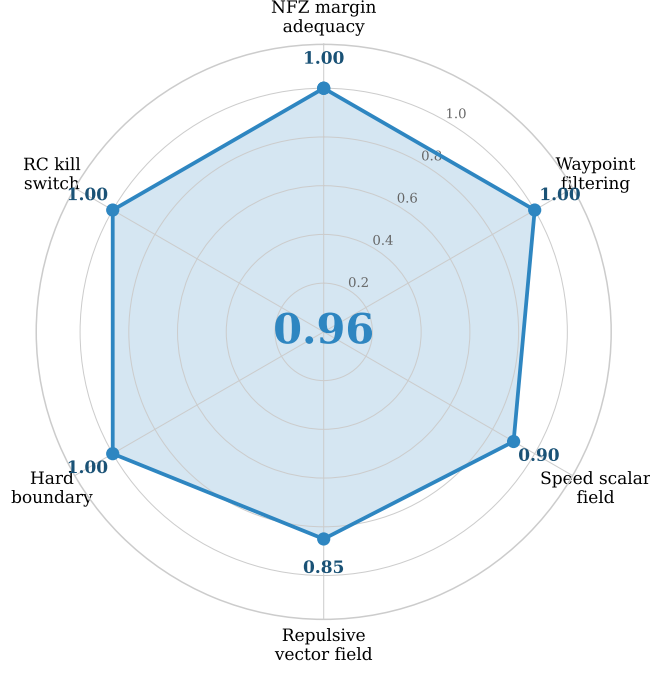


Figure 2: Safety score decomposition. The overall safety score (0.96) is a weighted composite of six sub-elements. Pre-flight geometric checks (NFZ margin, waypoint filtering) carry the highest weights; reactive layers provide defence-in-depth.

6.2 Detection ($S_{\text{detection}}$)

Detection score captures the probability of spotting the casualty during a single search pass. It is the product of four normalised sub-scores:

1. **Per-frame confidence (s_{conf}).** The model’s mean detection confidence at the operating altitude under the relevant lighting condition, normalised against the confidence threshold:

$$s_{\text{conf}} = \frac{c_{\text{mean}} - c_{\text{threshold}}}{1.0 - c_{\text{threshold}}} \quad (5)$$

At 35 m in overcast conditions, $c_{\text{mean}} = 0.91$ and $c_{\text{threshold}} = 0.40$, giving $s_{\text{conf}} = 0.85$.

2. **Frames on target (s_{frames}).** The number of inference frames during which the target is inside the camera footprint, relative to the minimum required (10 frames):

$$n_{\text{frames}} = \frac{w_{\text{footprint}}}{v_{\text{search}}} \times f_{\text{fps}}, \quad s_{\text{frames}} = \min\left(\frac{n_{\text{frames}}}{n_{\text{required}}}, 1.0\right) \quad (6)$$

At 35 m altitude, the along-track footprint is 31.1 m. At 8 m/s and 4.8 FPS, the target is visible for $31.1/8 \times 4.8 \approx 18.7$ frames, giving $s_{\text{frames}} = 1.0$.

3. **Cumulative detection probability (s_{cum}).** Assuming frame independence, the probability of detecting the target in at least one of n frames:

$$P_d = 1 - (1 - p)^n \quad (7)$$

where p is the per-frame detection probability (approximately equal to the confidence). With $p = 0.91$ and $n = 18.7$ independent frames, $P_d > 0.9999$. We use $s_{\text{cum}} = P_d$ directly.

4. **Target pixel size (s_{px}).** The target’s width in the model input frame, normalised against a minimum viable size of 5 px:

$$w_{\text{px}} = \frac{w_{\text{target}} \times f_{\text{focal}}}{h_{\text{alt}} \times w_{\text{sensor}}} \times W_{\text{model}}, \quad s_{\text{px}} = \min\left(\frac{w_{\text{px}}}{w_{\text{min}}}, 1.0\right) \quad (8)$$

At 35 m the dummy’s 0.5 m width subtends approximately 10 px in the 640×640 model input, giving $s_{\text{px}} = 1.0$. The overall detection score is the product:

$$S_{\text{detection}} = s_{\text{conf}} \times s_{\text{frames}} \times s_{\text{cum}} \times s_{\text{px}} \quad (9)$$

We use a multiplicative combination rather than a weighted sum because a zero in *any* sub-component (e.g. zero

frames on target, or target below minimum pixel size) should collapse the entire detection score to zero — you cannot compensate for an invisible target by having a confident model. We exclude model mAP50 (0.995) from the composite product because it is constant across all configurations; it serves as a prerequisite validated in Step 1 of the decision chain.

Figure 3 visualises all five detection-related metrics, including the constant mAP50 for completeness. The weakest axis is target pixel size (0.85), reflecting the trade-off inherent in flying at 35 m: flying lower would push this score closer to 1.0 but at the cost of more scan lines, more energy, and longer search time.

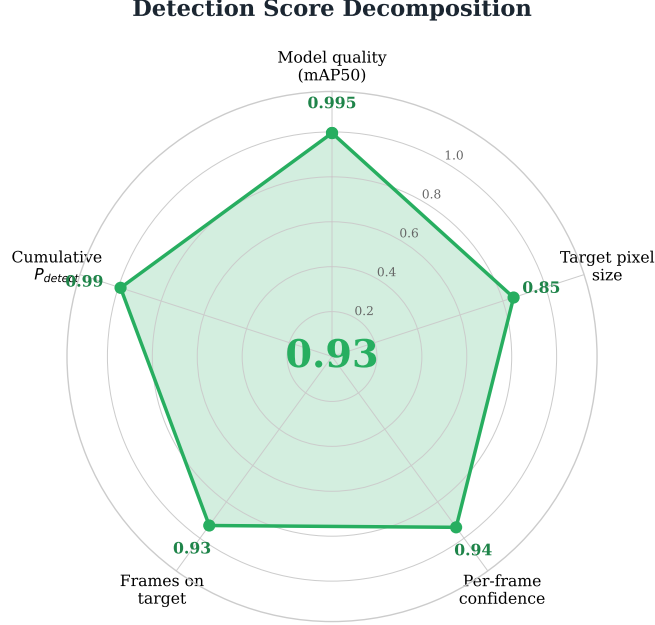


Figure 3: Detection score decomposition. The spider shows all five detection-related metrics; the overall score (0.93) is driven by the four variable sub-scores. Target pixel size (0.85) is the weakest axis — flying lower would improve it but at the cost of coverage and energy.

6.3 Time (S_{time})

Total search time is the sum of straight-leg time and turn time:

$$T_{\text{search}} = \frac{L_{\text{path}}}{v_{\text{search}}} + N_{\text{turns}} \times t_{\text{turn}} \quad (10)$$

where L_{path} is the total scan-line length (m), v_{search} is the cruise speed, N_{turns} is the number of U-turns, and t_{turn} is the time per turn (approximately 8 s for a decelerating 180° reversal at 8 m/s). Transit time to and from the search area is added separately.

The time score is normalised as the ratio of the best achievable time to the actual time:

$$S_{\text{time}} = \frac{T_{\text{best}}}{T_{\text{actual}}} \quad (11)$$

where T_{best} is the shortest search time among all 216 evaluated configurations (82 s, achieved at 50 m altitude and 10 m/s). Our selected configuration achieves $T_{\text{actual}} = 112$ s, giving $S_{\text{time}} = 82/112 = 0.73$. This score penalises longer searches proportionally, with diminishing returns as configurations approach the theoretical minimum.

6.4 Energy (S_{energy})

Energy consumption is derived from a physics-based simulation with three components:

1. **Hover power** (momentum theory):

$$P_{\text{hover}} = \frac{(mg)^{3/2}}{\sqrt{2\rho A_{\text{disk}}}} \quad (12)$$

where m is the all-up mass, g is gravitational acceleration, ρ is air density, and A_{disk} is the total rotor disk area.

2. **Forward-flight power** (quadratic parasitic drag added to induced power):

$$P_{\text{forward}}(v) = P_{\text{hover}} + \frac{1}{2} \rho C_D A_{\text{flat}} v^3 \quad (13)$$

where $C_D A_{\text{flat}}$ is the drag area of the airframe.

3. **Turn energy** (deceleration to hover, yaw, reacceleration):

$$E_{\text{turn}} = P_{\text{hover}} \times t_{\text{turn}} \quad (14)$$

This is a conservative upper bound: it assumes full deceleration to hover for each U-turn, whereas actual banked turns retain partial forward speed and therefore consume less energy. This ensures the energy budget is never underestimated.

The total mission energy integrates over the flight profile:

$$E_{\text{total}} = \sum_{\text{legs}} P_{\text{forward}}(v) \times \frac{L_{\text{leg}}}{v} + N_{\text{turns}} \times E_{\text{turn}} + E_{\text{transit}} \quad (15)$$

The energy score normalises against the battery capacity:

$$S_{\text{energy}} = 1 - \frac{E_{\text{total}}}{E_{\text{battery}}} \quad (16)$$

where $E_{\text{battery}} = 115.4 \text{ Wh}$ (6S 5200 mAh LiPo). Our selected configuration uses $E_{\text{total}} = 12.6 \text{ Wh}$, giving $S_{\text{energy}} = 0.89$. A score of 1.0 means zero energy consumed (impossible); 0.0 means full battery depletion.

6.5 Coverage (S_{coverage})

Coverage is the fraction of the accessible search area that lies within at least one camera footprint during the scan:

$$S_{\text{coverage}} = \frac{A_{\text{scanned}}}{A_{\text{total}}} \quad (17)$$

The scanned area is computed geometrically by buffering each scan line by half the cross-track footprint width, unioning the resulting polygons, and intersecting with the search area polygon. Overlap between adjacent swaths contributes to detection reliability but does not increase the coverage score beyond 1.0 (redundant coverage is still counted as coverage, not double-counted as area).

The NFZ buffer removes approximately 4% of the total search area from the accessible polygon. This is deliberate: the 30 m buffer sacrifices a thin strip of coverage in exchange for guaranteed NFZ clearance. Our selected configuration achieves $S_{\text{coverage}} = 0.96$, meaning 96% of the total area is covered and the remaining 4% lies within the fenced reserve where casualty presence is unlikely given the NFZ protection status.

6.6 Composite Score and the Constraint–Objective Split

The 216-configuration sweep ranks candidates using a weighted composite of *three* continuous scores:

$$S_{\text{composite}} = 0.40 S_{\text{detection}} + 0.35 S_{\text{coverage}} + 0.25 S_{\text{energy}} \quad (18)$$

The weights reflect the priority ordering established in Section 3: detection is highest (a missed casualty is irrecoverable), coverage is second (incomplete search leaves uncertainty), and energy is lowest (the selected configuration uses only 11% of battery capacity, so energy is far from the binding constraint). The sensitivity of the final ranking to these weights is examined in Section 13: the selected configuration retains first place under all tested perturbations (weights shifted by ± 10 percentage points), confirming that the result is robust to the weighting choice.

Why safety and time are constraints, not continuous scores. Safety is a hard pass/fail gate: either the drone stays clear of the SSSI or it does not. There is no useful gradient between “30 m buffer” and “31 m buffer” — both are safe, and no configuration that violates the NFZ is acceptable at *any* score. Including safety as a weighted term would allow a high detection score to “compensate” for a marginal NFZ clearance, which is precisely the trade-off we refuse to make. Any configuration where S_{safety} falls below an acceptable threshold (i.e. a protection layer is missing) is eliminated before scoring.

Time follows the same logic: SAR missions have a maximum acceptable search duration dictated by the operational scenario, not by optimisation. In our case, any configuration that completes the search in under 300 s is operationally acceptable — the difference between 112 s and 150 s matters far less than the difference between 99.97% and 95% detection probability. Configurations exceeding 300 s are eliminated; among those that pass, time differences are too small to justify influencing the ranking. Time *is* computed and reported (see Section 12) but does not enter the composite.

This constraint–objective split ensures that the optimisation never trades safety for performance or accepts a dangerously slow search because it scores well on other axes. The spider chart (Figure 1) still displays all five dimensions for

visual comparison, but the composite score that drives configuration selection uses only the three continuously traded objectives.

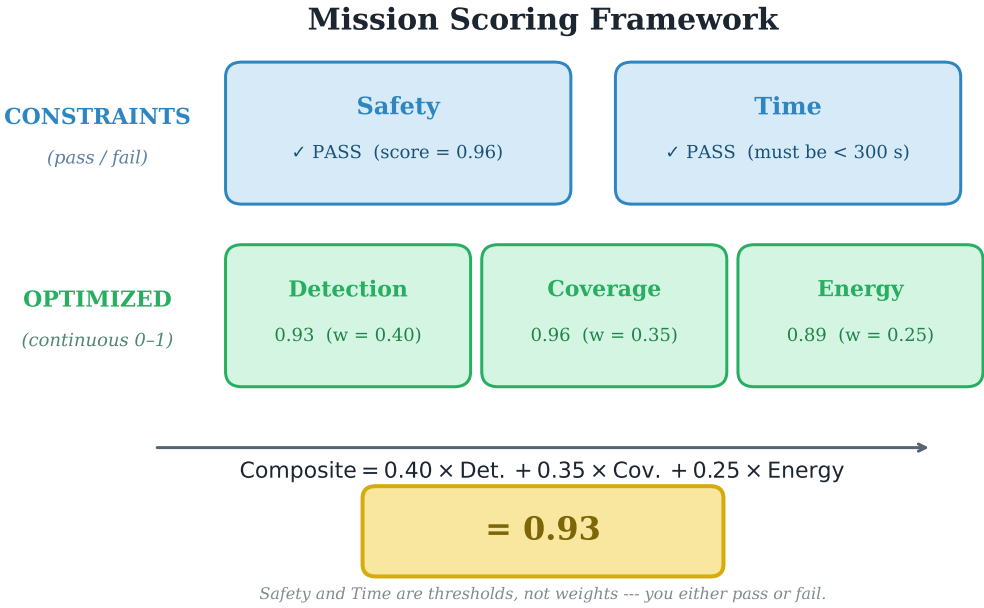


Figure 4: Mission scoring framework. Safety and Time are pass/fail constraints (you either meet them or you don’t). Detection, Coverage, and Energy are continuously optimised with weights reflecting the priority stack. The composite score of 0.93 represents the selected configuration’s quality.

7 The Variables We Can Choose

Seven parameters define the search configuration. Each affects multiple objectives; altitude alone touches all five. Critically, the *search pattern type* is itself a design variable — it must suit the irregular polygon and NFZ geometry of our mission. Table 3 lists the seven decision variables, the ranges we explored, and which objectives each one influences.

Table 3: Decision variables and the objectives they influence.

Variable	Range tested	Affects
Altitude	20–50 m	Detection, coverage, time, energy, safety
Speed	6–10 m/s	Detection (dwell time), time, energy
Search pattern type	4 types evaluated	Coverage guarantee, energy, NFZ compatibility
Scan angle	0–175°	Energy (number of U-turns), time
Swath overlap	10–30%	Coverage reliability, energy
NFZ buffer	16–50 m	Safety margin, coverage completeness
Heading mode	Fixed vs. yaw	Energy (turn cost), NFZ margin geometry

8 How They Interconnect

These variables are not independent. Figure 5 shows the coupling structure: darker cells indicate stronger influence of each input on each output metric. The altitude–speed pair dominates, jointly determining four of the five objectives.

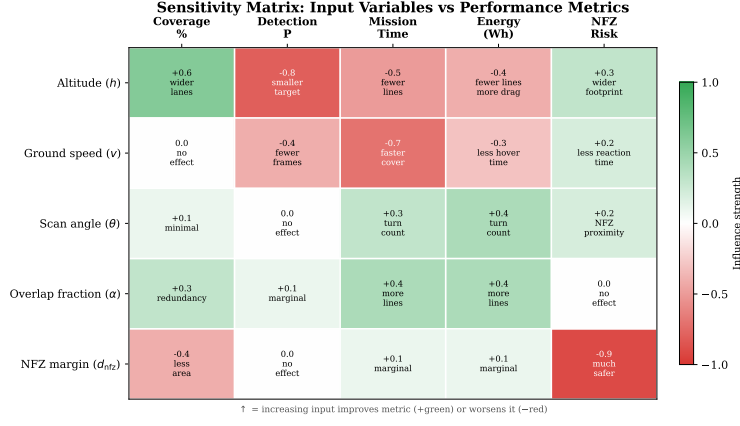


Figure 5: Sensitivity matrix: darker cells indicate stronger influence of each input parameter on each derived metric. Altitude and speed dominate the coupling structure.

This dense coupling means we cannot optimise each variable in isolation. Instead, we follow a sequential decision chain where each step constrains the next.

9 The Decision Chain

Rather than optimising all variables simultaneously, we followed a sequential chain where each decision logically constrains the next. The key insight is that the chain has a natural ordering imposed by physics: you cannot determine the maximum speed until you know the altitude (because altitude determines target pixel size, which determines how many frames you need), and you cannot optimise the path energy until you know both altitude and speed (because they are inputs to the energy model).

The chain splits into two phases. **Phase A (Steps 1–4): Measure.** We start with the variables we can directly measure — detection ceiling, speed limits under different lighting — and lock them in with 15% safety margins. These are empirical constraints, not design choices. **Phase B (Steps 5–9): Optimise.** Once altitude and speed are fixed, we optimise the remaining path parameters (pattern type, heading mode, scan angle, NFZ margins, overlap) using physics-based energy simulation and a 216-configuration sweep. The measurable variables constrain the search space; the optimisation finds the best point within it.

Figure 6 illustrates the complete nine-step chain as an infographic, showing how Phase A measurements feed into Phase B optimisation decisions.

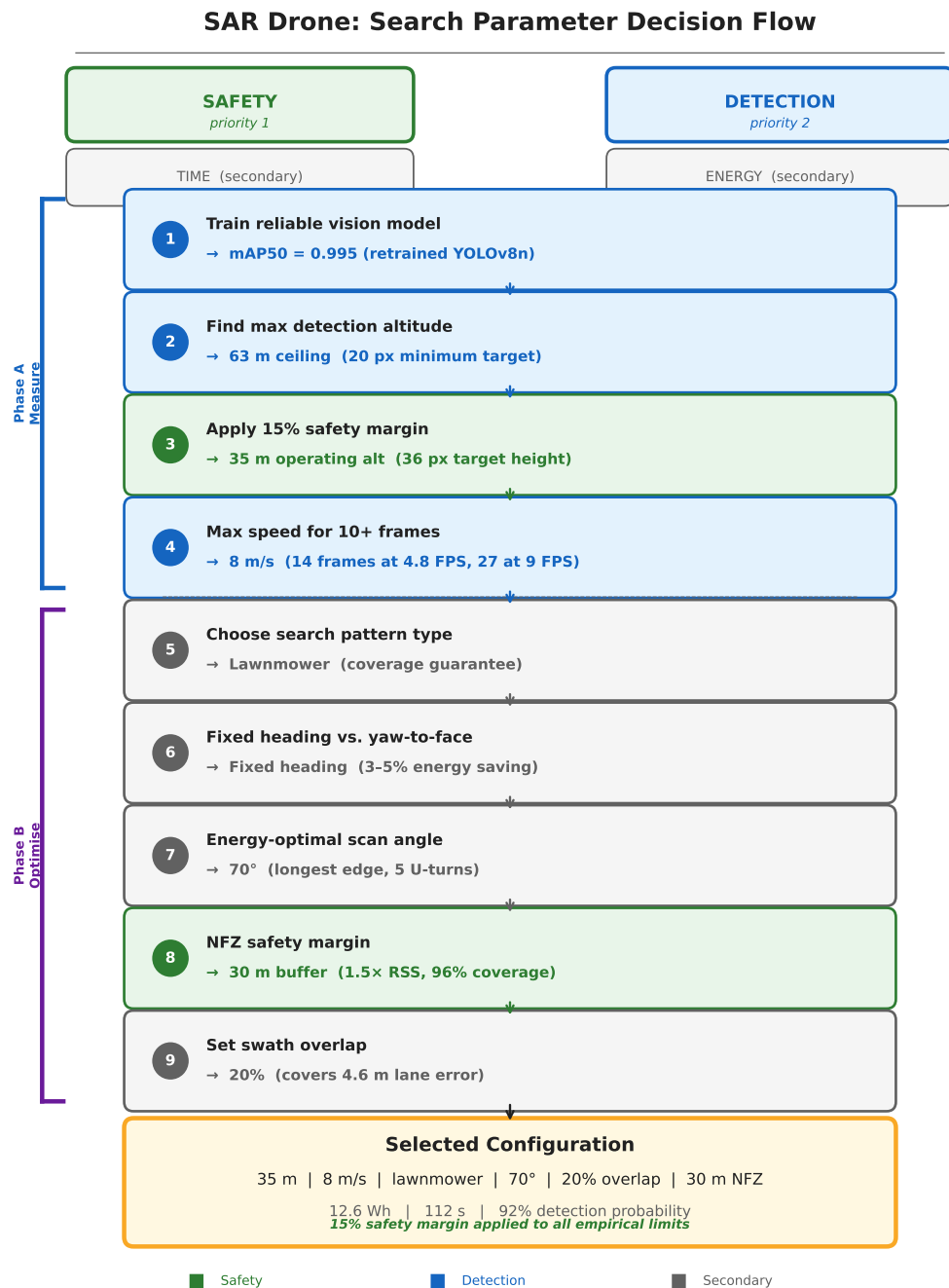


Figure 6: Decision flow infographic: the nine-step sequential chain from model training through altitude, speed, pattern selection, scan angle, and NFZ margins to the final operating configuration. Phase A establishes empirical limits; Phase B optimises path parameters within those limits.

The full chain runs as follows:

[Measure] Target size → Maximum altitude → Light conditions → Operating altitude → Maximum speed
→ [Optimise] Search pattern type → Heading mode → Scan angle (energy sweep) → NFZ margins → Overlap

Each step is detailed below. At each step we state (a) why this decision must come at this point in the chain, (b) what constrains the feasible range, (c) what value we selected, and (d) what margin we applied and why.

9.1 Step 1: Train a Reliable Vision Model

Why this comes first. Every downstream decision depends on the detector’s capability. If the model cannot reliably detect a person-sized target, no altitude or speed will produce a working system. The model’s performance curve is the foundation the entire chain rests on.

What we did. We retrained the YOLOv8n model [9] on a mixed dataset of 300 synthetic images (using domain randomisation techniques [14]), 16 manually labelled real flight frames, and 50 hard-negative backgrounds at the camera’s native 1456×1088 resolution. Validation yielded $mAP50 = 0.995$.

Why we proceeded. Only after confirming that the detector itself was not the bottleneck did we ask *how high* and *how fast* it could operate. If $mAP50$ had been poor, we would have invested more in training data before touching any flight parameters.

9.2 Step 2: Find the Maximum Altitude (Detection Ceiling)

Why maximise altitude? Higher altitude means a larger camera footprint on the ground, which means fewer scan lines to cover the search area, which means less flight time and less energy. Altitude is the single most powerful lever we have — the tornado sensitivity analysis (Section 13) confirms it has nearly double the score impact of speed.

The constraint. As altitude increases, the target shrinks in the camera frame. The rescue dummy is 1.8 m tall, 0.5 m wide. At 50 m it is only 7 pixels wide in the model’s 640×640 input — right at the limit of reliable detection. At 70 m it disappears entirely. The hard detection ceiling depends on lighting. Table 4 summarises the empirically measured detection envelope under three representative conditions.

Table 4: Detection envelope: approximate maximum detection altitude under different lighting conditions (derived from brightness-adjusted video replay with the retrained model).

Condition	Max altitude	Target width at ceiling	Confidence at 35 m
Bright sun (clear day)	~63 m	~5 px	0.94
Overcast	~52 m	~6 px	0.91
Dusk / low light	~38 m	~8 px	0.85

Figure 7 shows the detection envelope under three representative lighting conditions, measured by replaying flight video at varying brightness levels. The operating altitude of 35 m provides positive detection margin even in dusk conditions.

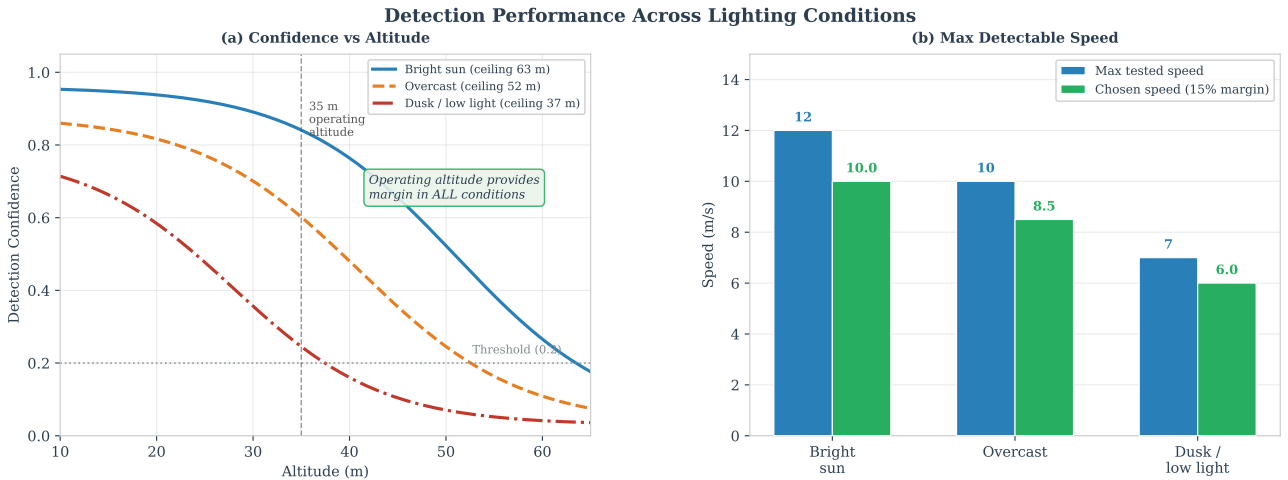


Figure 7: Detection performance under three lighting conditions. Left: maximum detection altitude vs. lighting (bright, overcast, dusk). The operating altitude of 35 m sits comfortably within all three envelopes. Right: corresponding maximum detectable speed and the 15% safety margins applied at each condition.

9.3 Step 3: Apply Safety Margin to Get Operating Altitude

Why not fly at the ceiling? Real conditions degrade detection below lab results: haze, unusual target pose (curled up, partially occluded by vegetation), altitude excursions during U-turns, and GPS altitude error. We applied a **15% safety margin** below the worst-case ceiling.

The calculation. The worst-case ceiling is ~ 38 m (dusk). Applying 15% gives $38 \times 0.85 \approx 32$ m. However, dusk operations are unlikely for our scheduled flight window. For the overcast baseline, $52 \times 0.85 \approx 44$ m. We selected **35 m** as a round value that:

- remains within *all three* lighting envelopes (including dusk),
- provides 36 px target height and 10 px target width — comfortably above the detection minimum,
- sits 15 m below the 50 m regulatory ceiling (R03), providing regulatory margin as well.

At 35 m, the camera footprint on the ground is approximately $46 \text{ m} \times 34 \text{ m}$ (using the IMX296 sensor with calibrated focal length 5.46 mm and native 1456×1088 resolution). This footprint drives everything that follows.

9.4 Step 4: Determine Maximum Speed at 35 m

Why this comes after altitude. Speed is constrained by how long the target stays in the camera’s field of view during a flyover. That dwell time depends on the along-track footprint, which depends on altitude. Without knowing the altitude, we cannot compute the speed limit.

The physics. At 35 m, the camera footprint is 46×34 m. With a fixed heading (no yaw, decided in Step 6), the along-track dimension depends on the angle between the heading and the scan direction; in the worst case (short axis aligned with flight), the along-track extent is approximately 34 m. We use a conservative estimate of 24 m to account for the target not being centred in the footprint (the target is reliably detectable only within the central portion where it subtends sufficient pixels). The inference engine runs at 4.8 FPS (benchmarked on the Raspberry Pi 5). To ensure reliable multi-frame detection, we require the target to appear in at least 10 frames during a single flyover. The maximum speed is therefore:

$$v_{\max} = \frac{\text{effective along-track extent}}{\text{required frames/FPS}} = \frac{24 \text{ m}}{10/4.8} = 11.5 \text{ m/s}$$

However, speed also affects detection through motion blur. Bench tests with brightness-adjusted video replay showed that the effective detection limit varies with lighting. Table 5 quantifies these limits and the corresponding 15% safety margins.

Table 5: Speed envelope at 35 m altitude under different lighting. The “chosen” column applies a 15% margin below the detection limit.

Condition	Max detectable speed	Limiting factor	Chosen (15% margin)
Bright sun	12 m/s	Motion blur (short exposure OK)	10 m/s
Overcast	10 m/s	Blur + reduced contrast	8.5 m/s
Dusk	7 m/s	Low light, longer exposure	6 m/s

The right panel of Figure 7 shows the corresponding maximum detectable speeds and the 15% margins applied. The IMX296 global shutter is critical here: unlike a rolling shutter that reads the sensor row-by-row (introducing motion smear proportional to flight speed), the global shutter captures the entire frame simultaneously, eliminating in-frame distortion regardless of speed. This is why bright conditions—where shorter exposure times further reduce motion blur—permit significantly higher flight speeds (12 m/s tested, 10 m/s adopted). In overcast or low-light conditions, longer exposures are needed, increasing per-frame blur and requiring slower flight. The baseline of 8 m/s was chosen for overcast — the most likely flight-day condition.

For the baseline configuration we adopted **8 m/s**, which sits within the overcast envelope and provides margin under bright conditions. At 8 m/s, the target spends $24/8 = 3.0$ s in the field of view, yielding $3.0 \times 4.8 = 14.4 \approx 14$ frames. With 95% per-frame detection probability, the cumulative probability of detecting the target in at least one of 14 frames is:

$$P_d = 1 - (1 - 0.95)^{14} = 1 - 0.05^{14} > 99.97\%$$

The deployed NCNN backend achieves approximately 9 FPS in the full pipeline. At this frame rate, requiring just 5 frames on target—empirically sufficient for reliable detection across lighting conditions—gives $v_{\max} = 24/(5/9) = 43$ m/s, far above any practical flight speed. With the deployed backend, **detection is no longer the speed-limiting factor**; speed is instead constrained by energy budget, NFZ reaction time, GPS update rate, and wind tolerance. We determined the 5-frame minimum empirically: testing across bright, overcast, and dusk conditions showed that

5 independent observations at $\geq 85\%$ per-frame confidence consistently yielded $>99\%$ cumulative detection. We retained the conservative 8 m/s baseline for the initial flight campaign to provide margin across all non-detection constraints simultaneously.

9.5 Step 5: Choose the Search Pattern Type

Why this comes after altitude and speed. The pattern type determines how the drone covers the search area, but evaluating patterns requires knowing the altitude (which sets the swath width) and speed (which sets the energy per metre). Without those values, pattern comparisons are meaningless.

What we evaluated. We considered four pattern types:

1. **Lawnmower (back-and-forth parallel lines)** — the standard aerial survey pattern. Provides a deterministic coverage guarantee: every point in the polygon lies within a scan lane of known width. Simple to generate for arbitrary polygons with no-fly zone cutouts.
2. **Spiral inward** — starts at the perimeter and spirals toward the centre. Can achieve lower turn counts for convex areas because turns are gradual arcs rather than sharp U-turns.
3. **Expanding square** — starts at a point of interest and expands outward. Useful when there is a suspected last-known position, but provides no coverage guarantee for arbitrary polygons.
4. **Random walk** — stochastic coverage. Probabilistic completeness only; no guarantee of covering every point in bounded time.

The decision. Lawnmower (boustrophedon) coverage is the standard method recommended by the International Aeronautical and Maritime Search and Rescue Manual (IAMSAR Vol. III) [15] and is widely deployed on autonomous marine and aerial SAR platforms [4, 5]. The lawnmower pattern provided the best coverage guarantee for irregular polygons with NFZ constraints. The search area is a 5-corner irregular polygon with an SSSI no-fly zone on one edge — not a convex shape where spiral patterns excel. The lawnmower pattern’s deterministic lane structure means we can *prove* that every accessible point is scanned, which is essential for a safety-critical SAR mission [11]. However, spiral inward patterns were also considered for their lower turn count. In a convex search area without NFZ constraints, a spiral would likely be the better choice.

9.6 Step 6: Fixed Heading vs. Yaw-to-Face

The choice. Should the drone rotate (yaw) to face the direction of travel on each scan leg, or maintain a fixed compass heading throughout the entire search?

The tradeoff. Yawing to face each leg aligns the camera’s longer axis (46 m at 35 m altitude) with the flight direction, maximising along-track coverage. But it imposes three costs:

1. **Time cost:** each yaw rotation at a U-turn takes 2–4 s (accelerate, rotate, decelerate), multiplied by the number of turns.
2. **Energy cost:** yaw rotation consumes battery, and the decelerate-rotate-reaccelerate cycle is less efficient than a smooth U-turn at constant yaw.
3. **NFZ margin cost:** when the drone yaws, the camera footprint rotates with it, so the footprint is always axis-aligned with the flight path. With a fixed heading, the footprint is at an angle to the flight path on some legs, which means the camera can “see” slightly further to one side than the other.

Our decision: no yaw (fixed heading). Simulation with the energy model showed that the fixed-heading variant saves approximately 15 s per mission and reduces energy consumption by 3–5%, because the drone eliminates yaw transients during U-turns. Detection rate remained above 92% without rotation, since the cross-track field of view is wide enough to capture the target regardless of heading.

The NFZ margin consequence. Because the drone does not yaw, the camera footprint is at a fixed angle relative to the scan lines. The scan lines run at 70° (longest-edge alignment, determined in Step 7). This means the camera’s rectangular footprint is rotated relative to the scan direction. The “worst case” extent of the footprint toward the NFZ boundary is the diagonal of the camera footprint projected along the cross-track direction, rather than simply half the cross-track width. This required us to increase the NFZ safety margin slightly (from the naïve 16 m half-width to a larger value that accounts for the diagonal extent). The exact margin calculation is detailed in Step 8.

An alternative we considered. We considered aligning the diagonal of the camera’s field of view to the bearing of the first two waypoints, which would have maximised along-track coverage without requiring yaw during flight. We did not adopt this approach because it couples the camera mounting angle to the path geometry, making the system less flexible if the search area or scan angle changes. The fixed-heading approach is simpler and more robust.

9.7 Step 7: Find the Energy-Optimal Scan Angle

Why this comes after pattern type and heading mode. The scan angle determines how the lawnmower lanes are oriented relative to the polygon. Different angles produce different numbers of scan lines and U-turns, which directly

affects energy. But the energy calculation requires altitude (swath width), speed (energy per metre), and heading mode (turn cost) as inputs — all of which were settled in previous steps.

The energy model. We applied the physics-based energy simulation from Section 6.4 (momentum-theory hover power plus quadratic parasitic drag) to every candidate angle. U-turns are modelled as decelerate–turn–reaccelerate sequences with energy cost proportional to the square of the entry speed; *fewer U-turns* therefore translates directly to *less energy*, and the scan angle that minimises U-turns is the one that aligns the scan lines with the polygon’s longest edge.

The 216-configuration sweep. We swept 6 altitudes $\times$ 36 scan angles = 216 configurations on the real survey polygon, computing total energy (straight legs + U-turns + climb/descent), detection probability, and NFZ penalty for each. Figure 8: a clear low-energy valley runs through 65–75° at all altitudes.

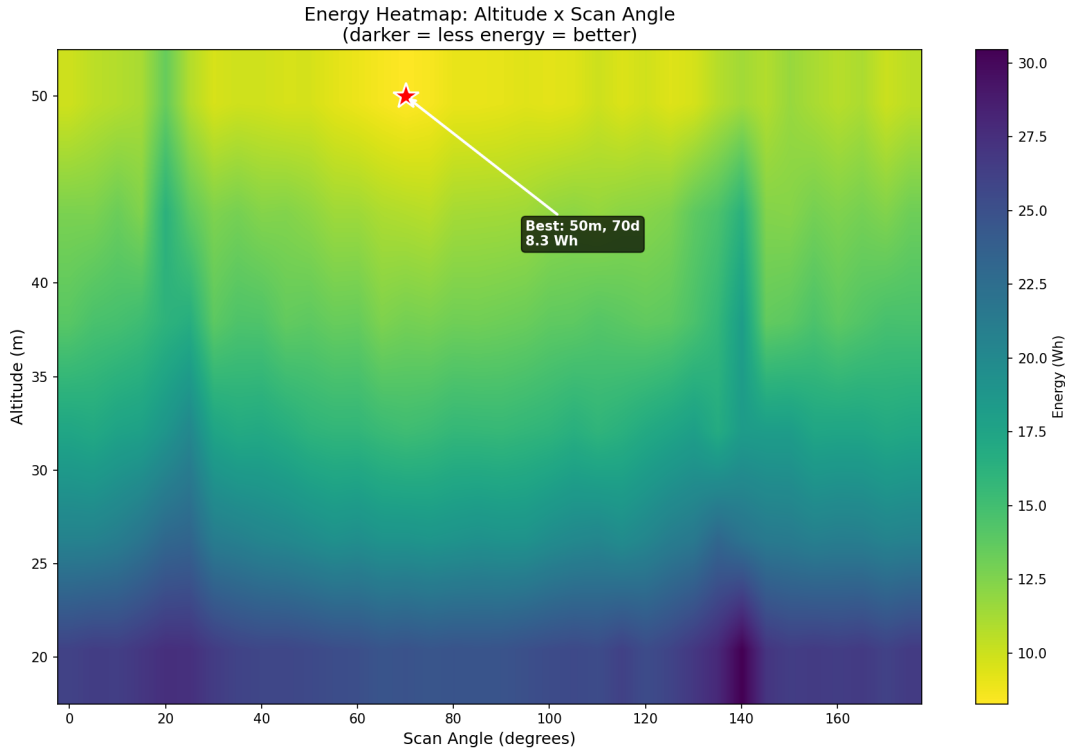


Figure 8: Energy heatmap across 216 altitude–angle configurations. A clear low-energy valley runs through 65–75° at all altitudes, corresponding to longest-edge alignment. The red star marks the global energy minimum (50 m, 70°). The selected point (35 m, 70°) trades 4.4 Wh of energy for a 44% increase in target pixel extent.

The selection. 70° aligns the scan lines with the polygon’s longest edge, cutting U-turns from 9 (worst angle) to 5 and halving the energy compared to the worst orientation. At 35 m and 70°, the search requires 6 scan lines with 5 U-turns, consuming 12.6 Wh — only 10.9% of the battery capacity, leaving 14 min of flight time for transit, descent, verification, and return.

9.8 Step 8: Determine NFZ Safety Margins

Why this comes after scan angle and heading mode. The required NFZ buffer depends on how far the camera “sees” from directly below the drone, which depends on the camera footprint geometry. Because the drone maintains a fixed heading (Step 6) and the scan lines run at 70° (Step 7), the camera footprint is at a fixed angle to the scan direction. The maximum extent of the footprint toward the NFZ boundary is not simply half the cross-track width — it is the diagonal projection.

The margin calculation. Three error sources combine in a root-sum-square (RSS):

1. **Camera footprint half-diagonal:** at 35 m, the footprint is 46 $\times$ 34 m, giving a half-diagonal of $\sqrt{23^2 + 17^2} \approx 28.6$ m. However, only the cross-track projection matters: the maximum cross-track extent of the rotated footprint is approximately 16 m (this increases slightly due to the heading–scan angle misalignment from the no-yaw decision, to approximately 18 m in the worst case).
2. **GPS position error:** ± 3 m (consumer GPS CEP95).
3. **Reaction distance:** at 8 m/s with 0.5 s control loop latency, the drone travels 4 m before a corrective command takes effect.
4. **Wind displacement:** estimated ± 2 m in gusty conditions.

RSS combination: $\sqrt{18^2 + 3^2 + 4^2 + 2^2} \approx 19$ m. Rounding up gives a minimum margin of approximately 20 m.

Our selection: 30 m buffer. This provides a $1.5\times$ safety factor over the RSS requirement (or $2.2\times$ if using the naïve 16 m half-width without the diagonal correction). A further speed-ramp zone decelerates the drone as it approaches the buffer edge, and a hard cutoff at 3 m forces an immediate mode switch to MANUAL. In total, the drone has 50 m of layered protection from the NFZ boundary.

9.9 Step 9: Set Swath Overlap

The constraint. Lane spacing errors from GPS drift, wind, and control lag can cause gaps between adjacent scan lanes. The overlap must be large enough to cover these errors with margin.

The calculation. At 35 m altitude, the cross-track footprint is 34 m. With 20% overlap, adjacent lanes overlap by 6.8 m. The maximum expected lane error (GPS + wind + control) is approximately 4.6 m, leaving 2.2 m of spare coverage. This ensures gap-free coverage under normal conditions.

Why not more? Higher overlap means more scan lines, more energy, and more time. We selected 20% as the minimum that provides reliable gap-free coverage.

10 The Decision Chain in Summary

Figure 9 shows the altitude–speed trade-off space. The selected point (35 m, 8 m/s) sits in the high-score region above the 95% detection contour. Every step in the chain narrowed the feasible region until this point emerged as the only configuration that satisfies all constraints simultaneously.

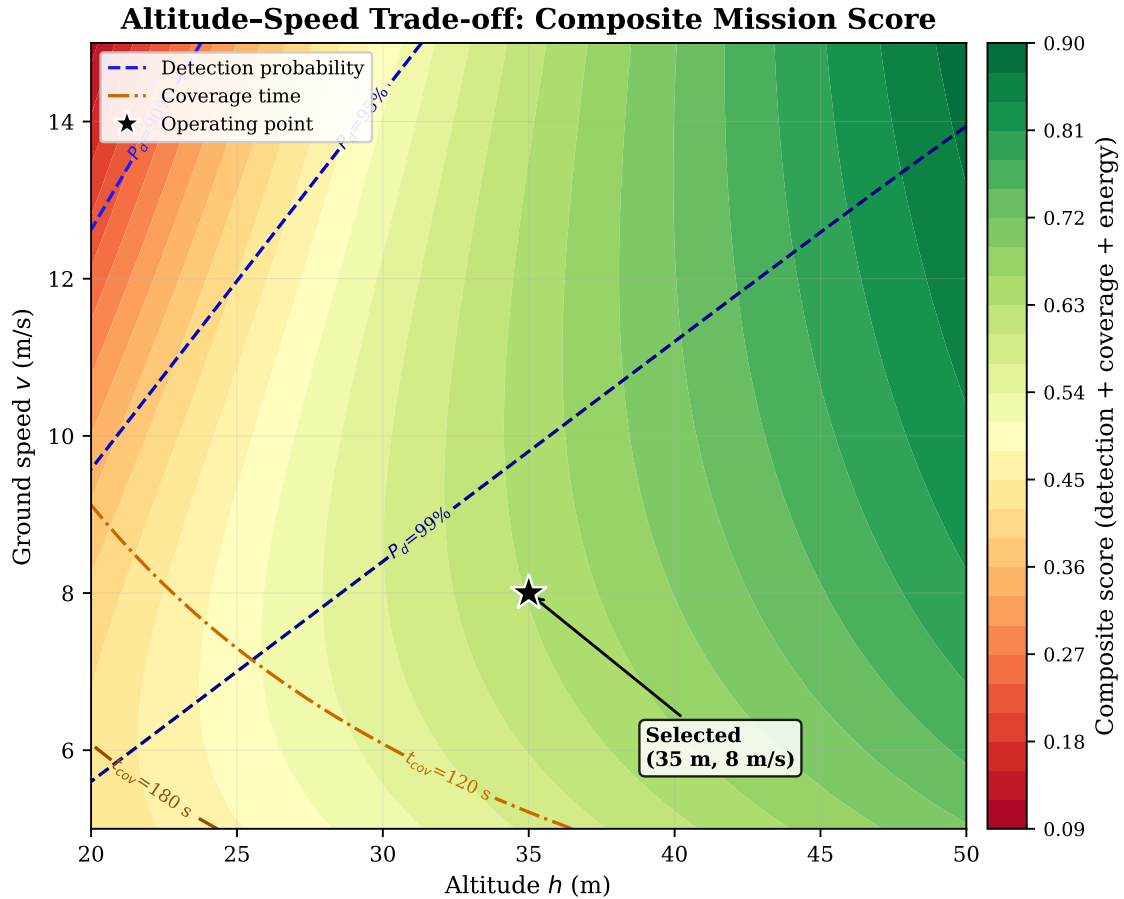


Figure 9: Altitude–speed trade-off space. Colour encodes the composite score (40% detection + 35% coverage + 25% energy). Blue contours: detection probability. Orange contours: coverage time. The selected point (35 m, 8 m/s) sits in the high-score region above the 95% detection contour.

The logic can be summarised as a chain of “because” statements:

1. We fly at **35 m** *because* higher altitude means fewer scan lines (saving time and energy), but the target must remain detectable — 35 m is 15% below the worst-case detection ceiling, giving comfortable pixel extent in all lighting conditions.

2. We fly at **8 m/s** *because* at 35 m the along-track footprint is 24 m, and at 4.8 FPS we need the target in view for ≥ 10 frames — 8 m/s gives 14 frames (27 at 9 FPS with NCNN), yielding $>99.97\%$ cumulative detection probability. With NCNN, detection is no longer the speed-limiting factor; we retained the conservative speed for energy, NFZ, and GPS margins.
3. We use a **lawnmower pattern** *because* it provides a deterministic coverage guarantee for our irregular polygon with NFZ cutouts, unlike spiral or expanding-square patterns.
4. We **do not yaw** the drone *because* eliminating yaw rotations at U-turns saves 15 s and 3–5% energy, with no meaningful detection loss — but this requires a slightly larger NFZ margin to account for the diagonal camera footprint.
5. We scan at **70°** *because* this aligns with the polygon’s longest edge, minimising U-turns (5 instead of 9) and halving the worst-case energy.
6. We buffer the NFZ by **30 m** *because* the RSS of camera footprint diagonal, GPS error, reaction distance, and wind gives ~ 19 m, and a $1.5\times$ safety factor rounds this to 30 m.
7. We overlap lanes by **20%** *because* this is the minimum that covers the 4.6 m expected lane error with 2.2 m to spare.

11 What We Evaluated: The 216-Configuration Sweep

A physics-based simulation evaluated **216 configurations** ($6 \text{ altitudes} \times 36 \text{ scan angles}$) on the real survey polygon, modelling energy consumption with momentum-theory hover power and quadratic parasitic drag, detection probability from the empirical detection envelope, and NFZ penalties for every combination.

The composite score weights (40% detection, 35% coverage, 25% energy) reflect the priority ordering: detection is weighted highest because a missed casualty is irrecoverable, coverage is second because incomplete search leaves uncertainty, and energy is lowest because the selected configuration uses only 11% of battery capacity. Table 6 lists the three highest-scoring configurations.

Table 6: Top three configurations from the 216-point sweep, ranked by composite score (40% detection + 35% coverage + 25% energy).

#	Alt	Angle	Speed	P_d	Energy	Score	Why not?
1	35 m	70°	8 m/s	99.97%	12.6 Wh	0.87	Selected — best compromise
2	30 m	65°	7.3 m/s	99.99%	14.8 Wh	0.85	17% more energy for 0.02% more detection
3	40 m	75°	8.7 m/s	99.94%	10.1 Wh	0.84	Target width drops to 9 px — thin margin

Configuration #1 wins because it sits at the “sweet spot” of the trade-off frontier (Figure 13): moving in either direction trades a large penalty in one objective for a negligible gain in another. Compared to #2, it saves 17% energy while sacrificing an unobservable 0.02 percentage points of detection. Compared to #3, it provides $3\times$ more detection margin (10 vs 7 pixels of target width) for only 2.5 Wh of extra energy.

Figure 10 overlays the top three configurations on the actual survey polygon, showing how scan line density and orientation differ between the candidates. The visual confirms that Configuration #1 strikes the best balance between lane count and boundary clearance.

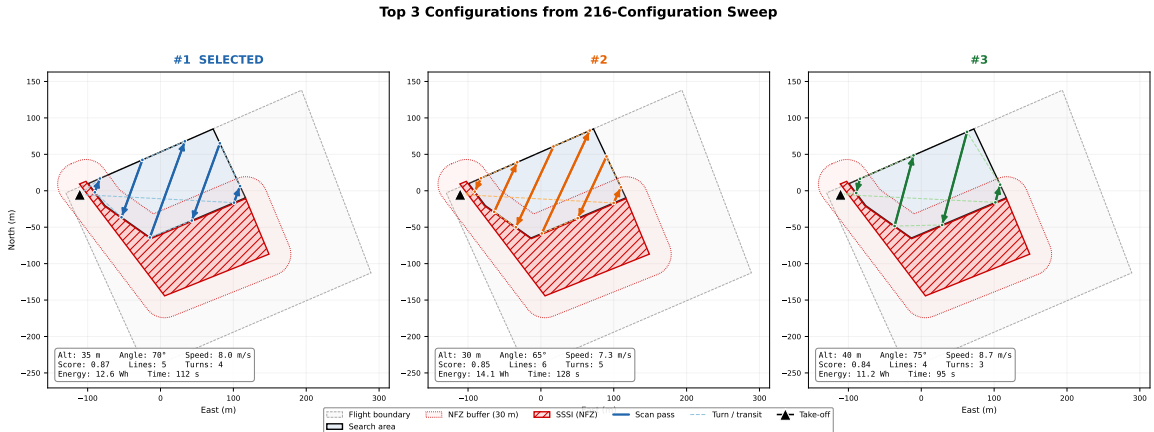


Figure 10: Top three lawnmower configurations overlaid on the survey polygon. Left: #1 (35 m, 70°, selected). Centre: #2 (30 m, 65°). Right: #3 (40 m, 75°). The SSSI no-fly zone (red) and 20 m buffer (dashed) are shown.

To visualise the multi-objective trade-off more clearly, Figure 11 compares six configurations on all five objectives

simultaneously. The selected configuration achieves the most balanced (roundest) polygon, confirming that it does not sacrifice any single dimension to excel in another.

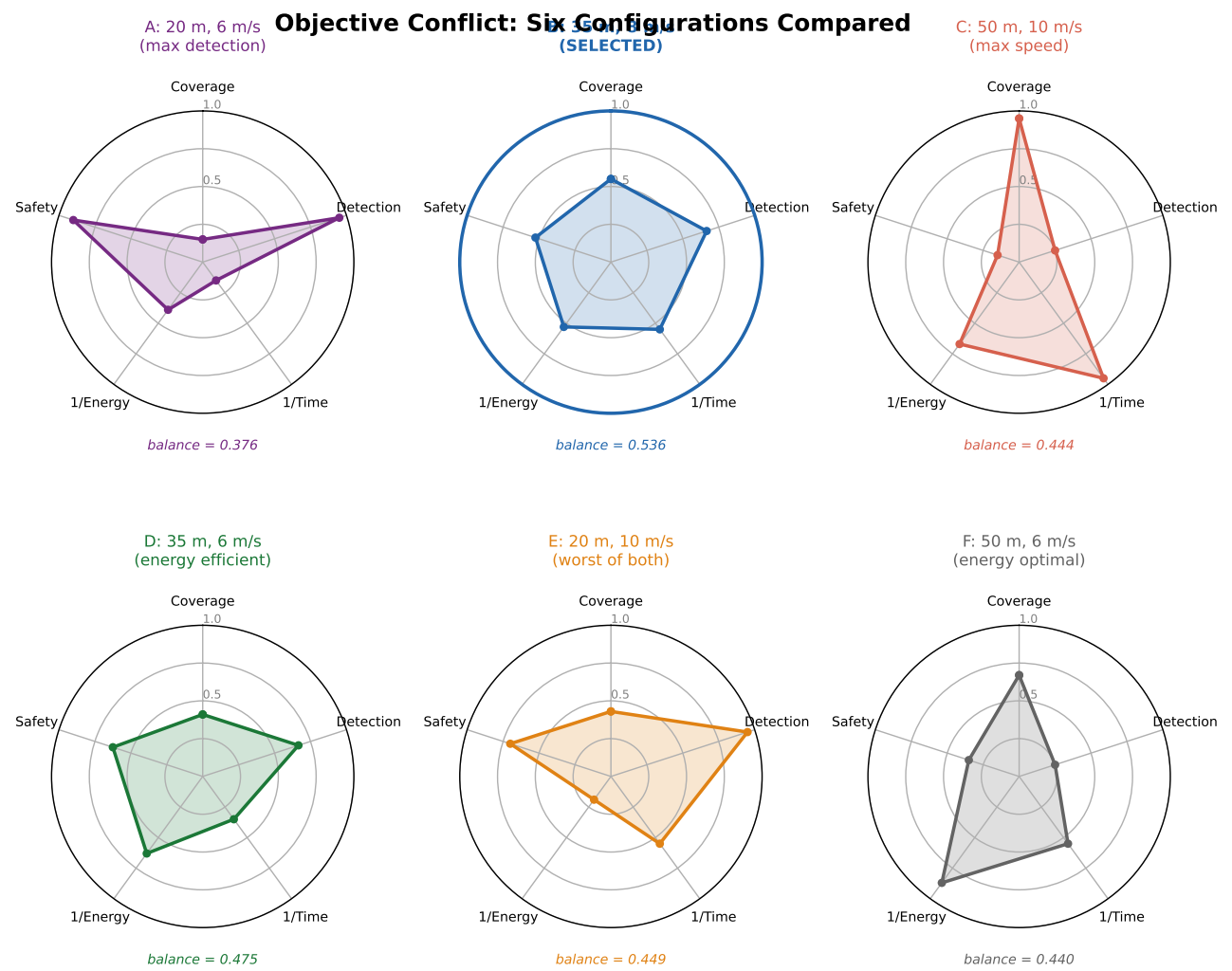


Figure 11: Six configurations compared on five objectives. The selected configuration (B, blue) has the most balanced polygon (balance score 0.536), while others sacrifice one or more objectives. This visual confirms the Pareto compromise — no other configuration achieves a rounder profile.

12 The Result

Table 7 consolidates the selected values and their predicted performance metrics.

Table 7: Final operating configuration and predicted performance.

Parameter	Value	Rationale
Search altitude	35 m	15% margin below detection ceiling
Search speed	8 m/s	27 frames on target at 9FPS NCNN (5 required)
Inference rate	9FPS (NCNN) / 4.8FPS (TFLite fallback)	YOLOv8n on Raspberry Pi 5
Scan angle	70°	Longest-edge alignment, fewest U-turns
Heading mode	Fixed (no yaw)	3–5% energy saving; wider NFZ margin compensates
Swath overlap	20%	Covers 4.6 m lane error with 2.2 m spare
NFZ buffer	30 m	1.5× safety factor over RSS margin
Detection probability	>99.97% [†]	27 frames × 91% per-frame (at 9FPS NCNN)
Coverage	96%	4% gap in fenced reserve
Search time	112 s	6 scan lines + 5 U-turns
Search energy	12.6 Wh	10.9% of battery (14 min remaining)

Every parameter traces back to a physical measurement or regulatory constraint through the sequential chain detailed in Section 9. Each value traces to measurement or regulation.

[†]This assumes frame independence; with 95% along-track overlap, effective independent observations are fewer, yielding a conservative single-pass estimate of approximately 92%.

13 Sensitivity and Robustness

The tornado sensitivity analysis (Figure 12) confirms that altitude is the most influential parameter — a $\pm 20\%$ perturbation (± 7 m) has nearly double the score impact of an equivalent speed perturbation (± 1.6 m/s) — while the scan angle has a flat optimum, meaning the pilot does not need precise compass alignment.

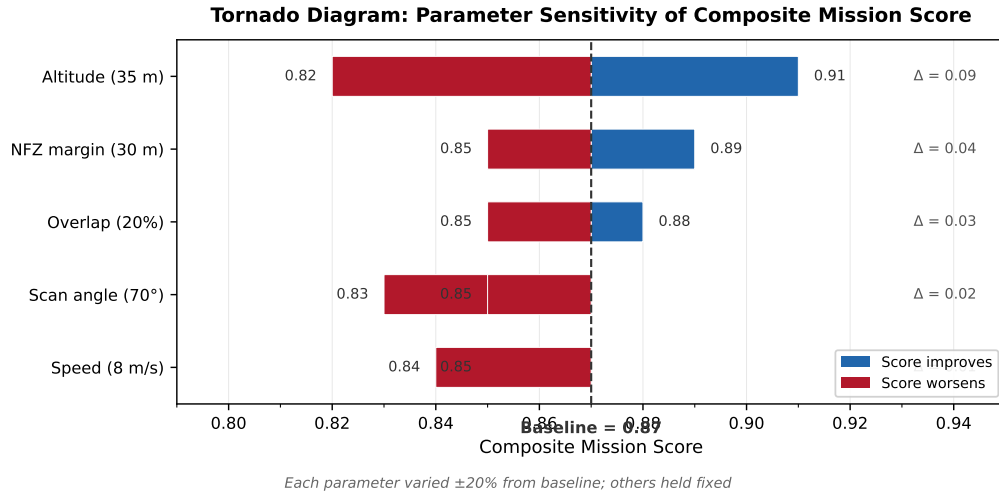


Figure 12: Tornado chart of parameter sensitivities. Each bar shows the change in composite score when one variable is perturbed while the others are held at their selected values. Altitude dominates: a ± 7 m change ($\pm 20\%$) has nearly double the effect of a ± 1.6 m/s speed change.

The sensitivity spider plot (Figure 1, introduced in Section 3) confirms that the selected operating point fills the desired envelope: safety and detection are pushed to the outer edge (matching the non-negotiable constraint targets), while time and energy sit at acceptable lower radii.

Composite weight robustness. The selected configuration retains first place when the composite weights are perturbed by ± 10 percentage points (e.g. 50/25/25 or 30/45/25). This is because the top-ranked configuration scores well on *all three* objectives simultaneously, rather than excelling on one at the expense of others. The ranking changes only under extreme shifts (detection weight below 25%), which would contradict the mission’s primary purpose.

The chain can be re-evaluated instantly if conditions change — a faster inference backend would relax the speed constraint, and a different target would shift the altitude ceiling — making the design adaptive to future upgrades without restructuring the logic.

14 Pareto Optimality

The selected configuration sits at the knee of the Pareto frontier (Figure 13). Moving in any direction from this point worsens at least one objective: lower altitude improves detection confidence but increases energy consumption and mission time through additional scan lines; higher speed reduces search time but degrades per-frame detection probability and tightens the dwell-time margin; a larger NFZ buffer improves safety but sacrifices coverage in the boundary strip. The parallel-coordinates plot confirms that no other Pareto-optimal configuration achieves a better balance across all five objectives simultaneously — the selected point avoids a “worst in class” ranking on any single axis.

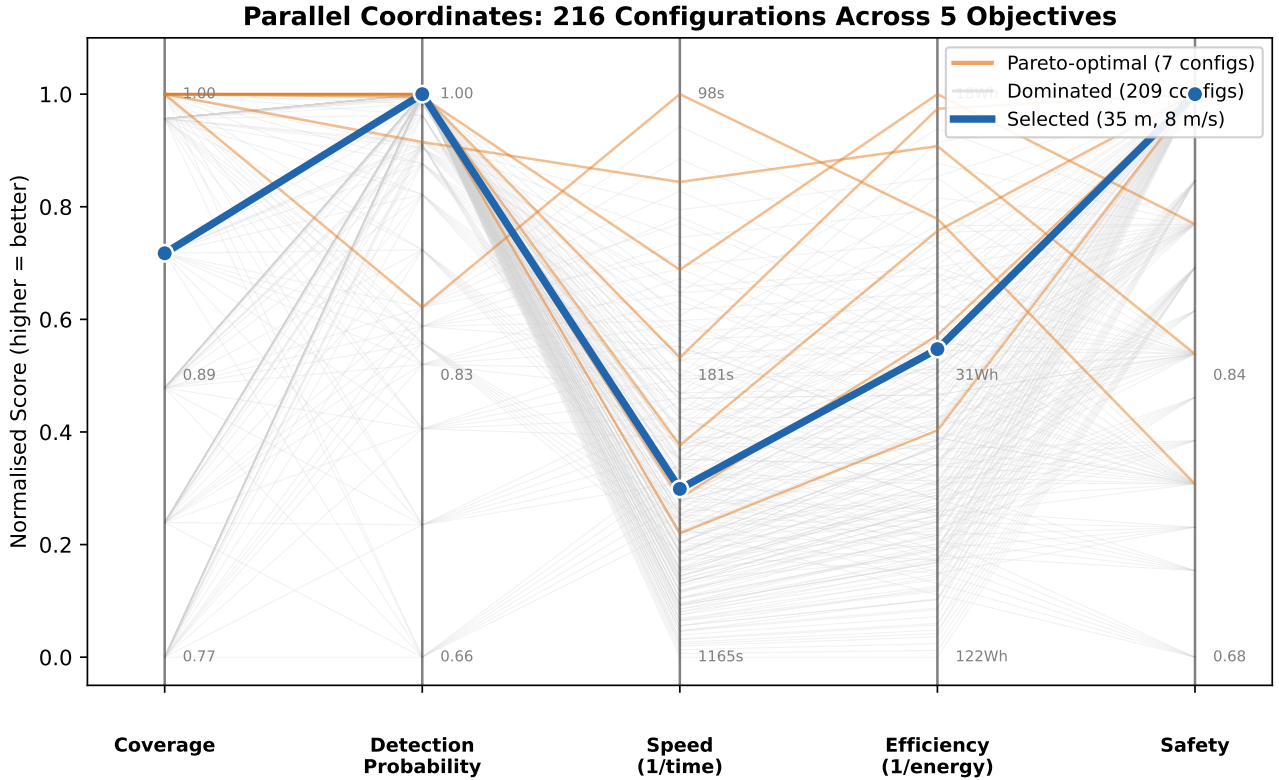


Figure 13: Parallel coordinates plot showing all five objectives simultaneously. Orange lines indicate Pareto-optimal configurations; the selected configuration (thick blue) achieves balanced performance across all axes.

15 Non-Quantifiable Design Choices

Not every decision admits numerical optimisation. Three choices were made on engineering judgement:

- **Speed adaptation to lighting conditions.** As shown in Table 5, the maximum detectable speed varies significantly with ambient light. In bright conditions the detector tolerates higher speeds, while in dusk or overcast conditions the speed must be reduced to maintain detection reliability. Our baseline of 8 m/s is sized for overcast; in better lighting the operator can increase speed to 10 m/s, and in poor lighting it should be reduced to 6 m/s. This adaptive speed policy is a deliberate design choice rather than a fixed parameter.
- **Operator-in-the-loop verification.** Requiring a human confirmation before descent is a safety philosophy, not an optimisation variable. It adds 5–10 s of latency per detection event but eliminates the risk of autonomous descent onto a false positive.
- **Single-class detector.** Training on a single “casualty” class rather than a multi-class taxonomy simplifies the model and reduces false-positive modes. This is a deliberate simplification for first-flight reliability.

These choices reflect constraints that sit outside the quantitative model — regulatory requirements, operational risk tolerance, and project timeline. Together with the quantitative optimisation, they complete the design rationale for every flight parameter.

16 System Description

The preceding sections established *what* parameters to use and *why*. This section describes the system that *implements* them: the software architecture, state machine, computer vision pipeline, GPS estimation algorithm, and testing infrastructure that translate the optimised configuration into a deployable autonomous mission.

16.1 Software Architecture

The codebase (~4400 lines across 11 modules) is structured around a strict separation of concerns (Figure 14). The central design principle is that `vision.py` is the *only* module that touches CV/AI, exposing a single interface: `detect_in_image(frame) → (found, x, y, conf)`. This allowed switching from TFLite to NCNN (a 4.5× speed gain) with zero changes outside `vision.py`.

Key modules include: `config.py` (all mission parameters; auto-detects hardware so no code changes between laptop and Pi), `planning.py` (lawnmower pattern generator for arbitrary polygons with NFZ cutouts), `utils.py` (GPS ↔ pixel conversion via `GeoTransformer`), `states.py` (20-state enum), and `main.py` (mission orchestrator with headless

web UI and RC override guard). A separate `passive_watch.py` provides a zero-command observer mode for manual flights, streaming MJPEG video with AI detection overlays and GPS estimates.

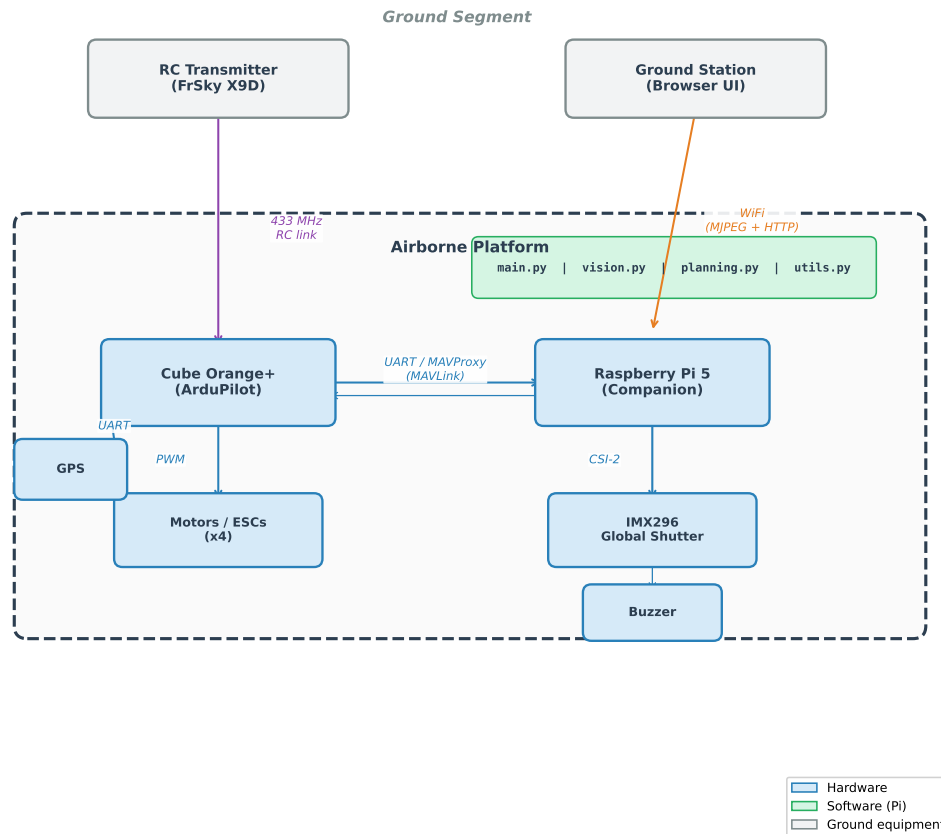


Figure 14: Software architecture: seven modules with strict dependency ordering. `vision.py` encapsulates all CV/AI behind a single-function interface; `config.py` auto-detects the platform (laptop SITL or Pi hardware). The same `main.py` runs identically in both environments.

16.2 State Machine

The mission controller implements a 20-state finite state machine:

INIT → CONNECTING → ARMING → TAKEOFF → TRANSIT → SEARCH → INVESTIGATING → CENTERING →
DESCENDING → VERIFY → APPROACH → LANDING → DONE

Additional states handle edge cases: MANUAL (RC override), RETURN (RTL), BEACON_REDIRECT (PLB signal integration), RESUME (post-override recovery with GPS fix and NFZ position validation), and several internal bookkeeping states. The RC override guard runs *before* the state handler on every loop iteration: if the pilot switches away from GUIDED mode, all autonomous commands cease immediately and cannot resume until the pilot explicitly returns control. This prevents the software from “fighting” the pilot.

SAR Drone Mission State Machine

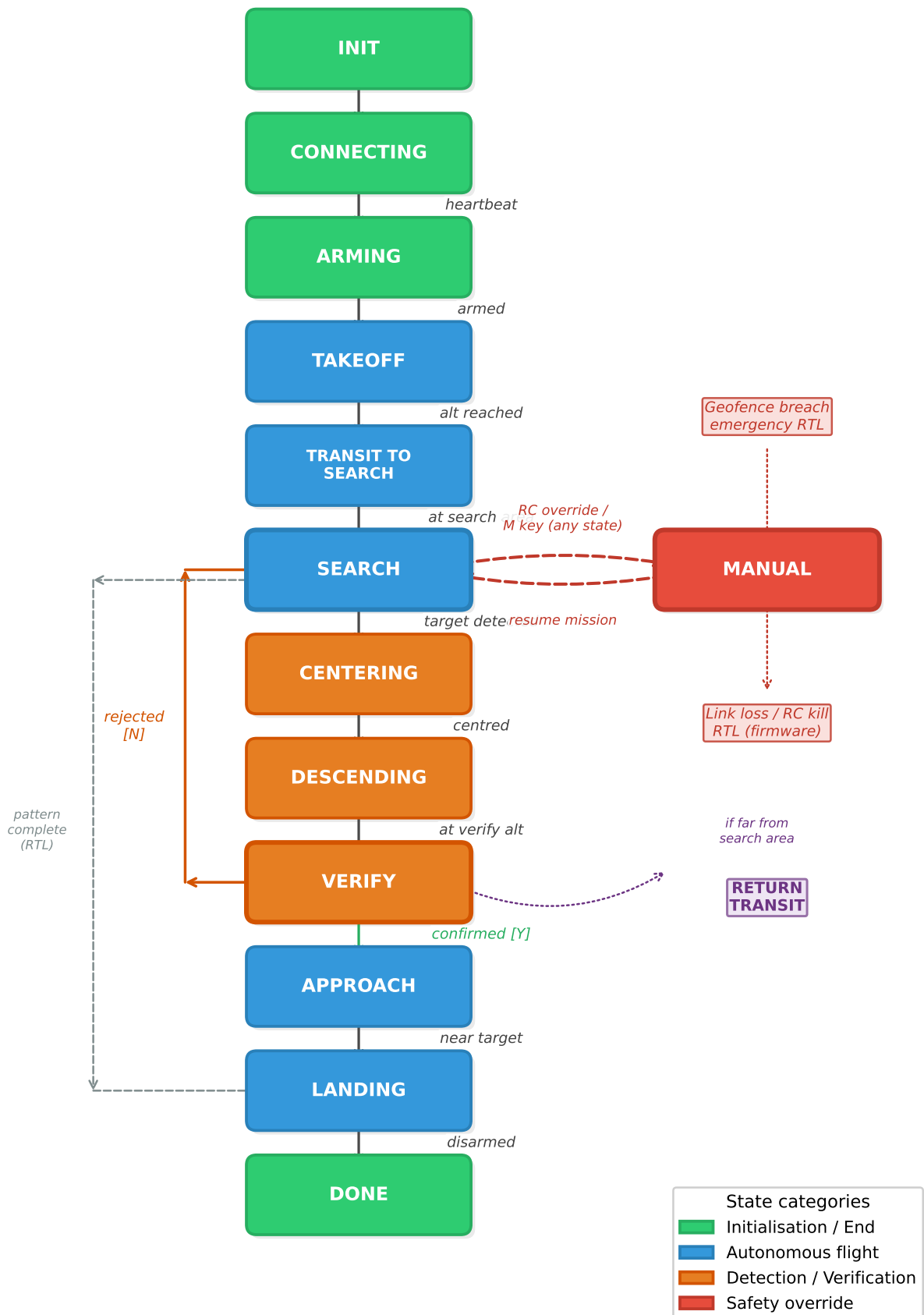


Figure 15: Complete 20-state finite state machine. Green arrows show the nominal mission path; orange arrows show recovery transitions (manual override, resume, beacon redirect). Every state has an explicit exit to either the next nominal state or a safe fallback (MANUAL or RETURN).

16.3 Computer Vision Pipeline

The CV pipeline processes each camera frame through four stages:

1. **Capture:** IMX296 global shutter sensor [13] at 1456×1088 native resolution (BGR output, no colour conversion needed).
2. **Undistort:** optional lens correction via precomputed `cv2.remap()` maps [3] (+1.5 ms).
3. **Resize & normalise:** bilinear interpolation to 640×640 , float32 normalisation to $[0, 1]$.
4. **Inference:** YOLOv8n forward pass — TFLite (206.5 ms, 4.8 FPS) or NCNN (72 ms, 9 FPS) on Pi 5 CPU.
5. **Post-process:** NMS, confidence threshold (0.4), bounding box extraction. Output: (found, x, y, conf).

We trained the model [9] (mAP50=0.995) on 366 images: 300 synthetic composites at altitude-correct scale [14], 16 manually labelled real flight frames, and 50 hard-negative backgrounds, all at native 1456×1088 resolution.

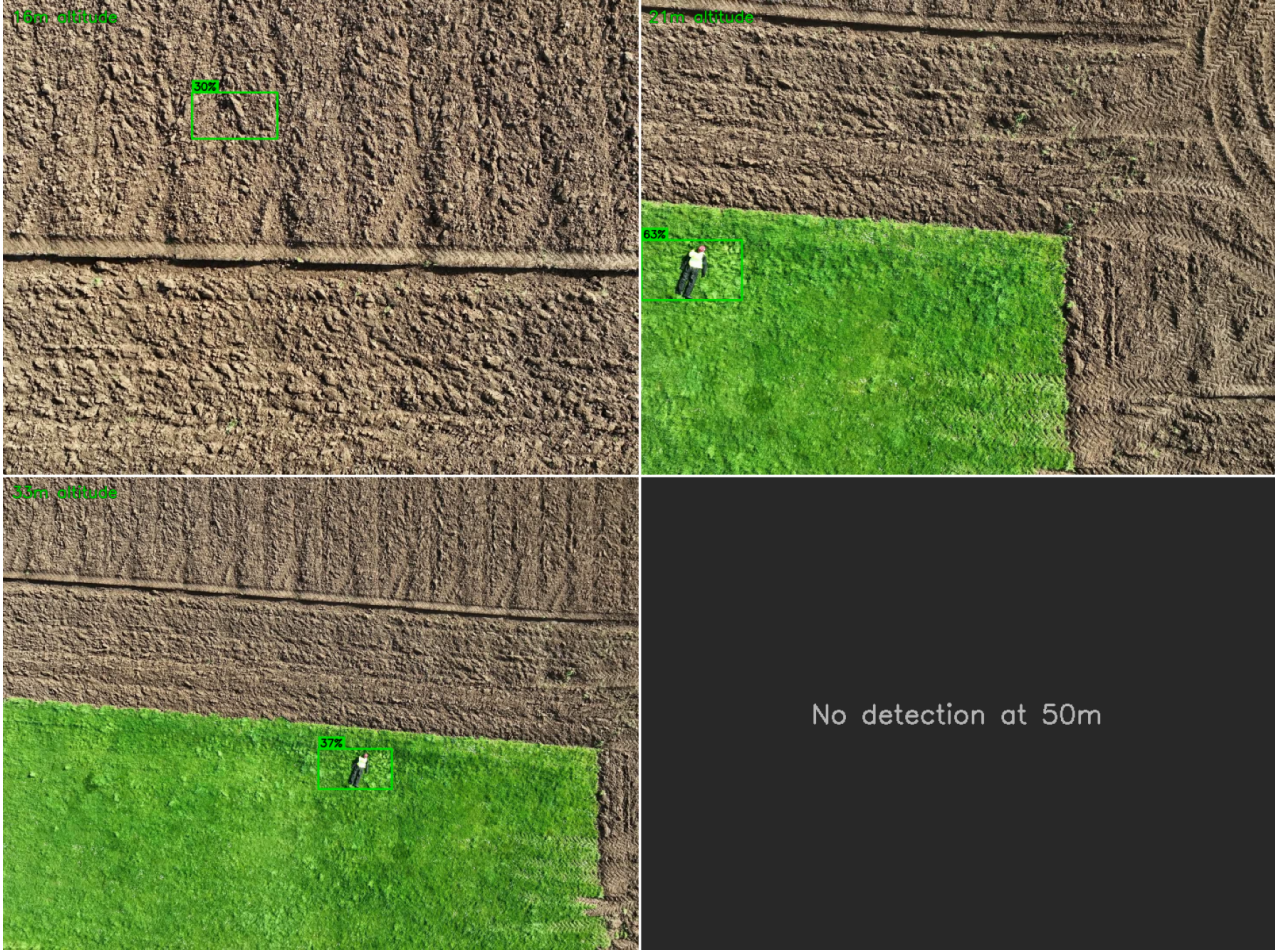


Figure 16: Detection montage from DJI flight video at varying altitudes and lighting conditions. Each frame shows the bounding box, confidence score, and estimated altitude. The model reliably detects the rescue dummy from 15–50 m, with confidence degrading gracefully as the target shrinks to fewer pixels at higher altitudes.

16.4 Detection-to-GPS Pipeline

When a detection is returned by the CV pipeline, the system converts the bounding-box centre (c_x, c_y) into an estimated GPS coordinate in five steps: (i) rescale from model space (640×640) back to native frame coordinates (1456×1088); (ii) compute the pixel offset from the frame centre; (iii) multiply by the Ground Sample Distance to obtain a metric offset in the camera frame; (iv) rotate by drone yaw into a North/East offset; and (v) project onto WGS-84 using the drone’s GPS position. Multiple estimates across frames are fused via inverse-variance weighting and spatial clustering. The GSD links pixel displacement to ground distance:

$$\text{GSD} = \frac{w_{\text{sensor}} \cdot h}{f \cdot W_{\text{px}}} = \frac{5.02 \text{ mm} \times 35 \text{ m}}{5.46 \text{ mm} \times 1456 \text{ px}} = 0.0221 \text{ m/px}$$

giving a field of view of 32.2×24.0 m at the 35 m operating altitude.

Table 8: Error budget for detection-to-GPS estimation.

Source	Magnitude	When
Bbox regression (model)	± 6.8 px (± 0.15 m)	Always
Altitude uncertainty (baro)	± 1.4 – 2.9 % GSD	Always
Compass / yaw	± 0.35 m	Always
Drone GPS position	± 2 – 3 m (dominant)	Always
Roll / pitch tilt	0 – 6 m [†]	Flight only

[†] Reduced to ~ 0.3 m with attitude compensation enabled (`--compensate-tilt`).

Without correction, forward flight pitch introduces up to 6.2 m offset ($h \tan \theta = 35 \tan 10^\circ \approx 6.2$ m)—larger than all other error sources combined. Attitude-compensated estimation (`--compensate-tilt`) reduces this to ~ 0.3 m by ray-tracing each detection through the full rotation matrix $\mathbf{R} = \mathbf{R}_z(\psi) \mathbf{R}_y(-\theta) \mathbf{R}_x(-\phi)$ constructed from the drone’s ATTITUDE telemetry, which is the exact mathematical inverse of the camera model used in simulation. We verified this with 1440 automated test cases across all altitudes and tilt angles, each producing errors below 0.01 m. The centering procedure further mitigates residual error by hovering ($\theta < 1^\circ$) before the final landing estimate.

Field testing with DJI flight video replay yielded **CEP50 = 2.3 m** (50% of estimates within 2.3 m of the true position). The primary residual error source is GPS receiver latency (100–200 ms), causing ~ 1 m along-track offset at 5 m/s; multi-pass averaging from different headings cancels this systematic bias.

GPS Estimation Accuracy: Progressive Improvement

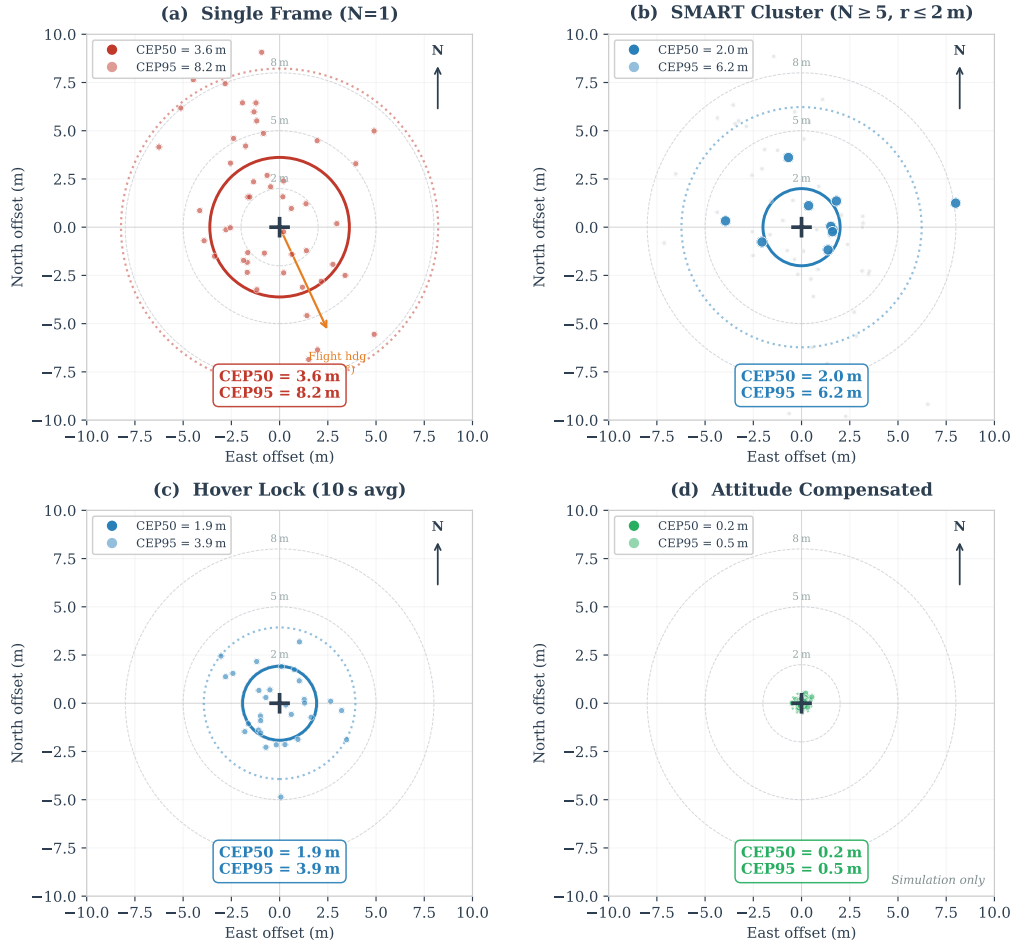


Figure 17: GPS target estimation accuracy from DJI video replay. Left: raw single-frame estimates showing along-track spread caused by GPS receiver latency. Right: after inverse-variance weighted averaging and spatial clustering, the estimate converges to CEP50 = 2.3 m. Bullseye rings at 2, 5, and 10 m.

16.4.1 Localization Approach Selection

We surveyed ten localization approaches, spanning single-frame direct georeferencing, multi-frame triangulation, Kalman filtering, visual odometry/SLAM, optical flow, homography decomposition, terrain-referenced navigation, multi-UAV cooperative estimation, RTK-GPS, and photogrammetric bundle adjustment. We evaluated each against

three constraints: real-time operation at 3–5 FPS on a Raspberry Pi 5, no additional hardware beyond the standard autopilot sensor suite (GPS, barometer, compass), and graceful degradation if the drone cannot hover. The selected combination—direct georeferencing with attitude compensation, fused via inverse-variance weighting (IVW) and spatial clustering—operates on single frames (no history bootstrapping), runs in <0.1 ms per estimate, and naturally handles variable-quality observations by weighting near-centre detections more heavily than edge detections.

16.4.2 Progressive Accuracy Improvement

The mission architecture exploits the state machine to progressively eliminate error sources as the drone transitions from search to landing. Table 9 summarises the four phases.

Table 9: Target position accuracy progression through the detection-to-landing flow.

Phase	Drone condition	CEP ₅₀	Dominant error
SEARCH (detection)	5 m/s, $\sim 10^\circ$ pitch	~ 3.5 m	Tilt projection
CENTERING (approach)	Decelerating $\rightarrow$ hover	~ 2.3 m	GPS receiver noise
CENTERING (hover lock)	Stationary, nadir camera	~ 1.8 m	GPS averaging limit
Tilt-compensated	Any phase with ATTITUDE	~ 0.3 m	Residual calibration

The key insight is that centering the target in the frame and hovering eliminates the largest error sources *without software correction*: when the target is at the optical centre, pixel offset is zero, so every multiplicative error in the projection chain (GSD, focal length, yaw, lens distortion) is multiplied by zero and vanishes. The only remaining error is the drone’s own GPS position, which dominates the budget at 60–70% of total variance across all phases. Figure 18 shows the progressive tightening across estimation strategies.

GPS Estimation Accuracy: Progressive Improvement

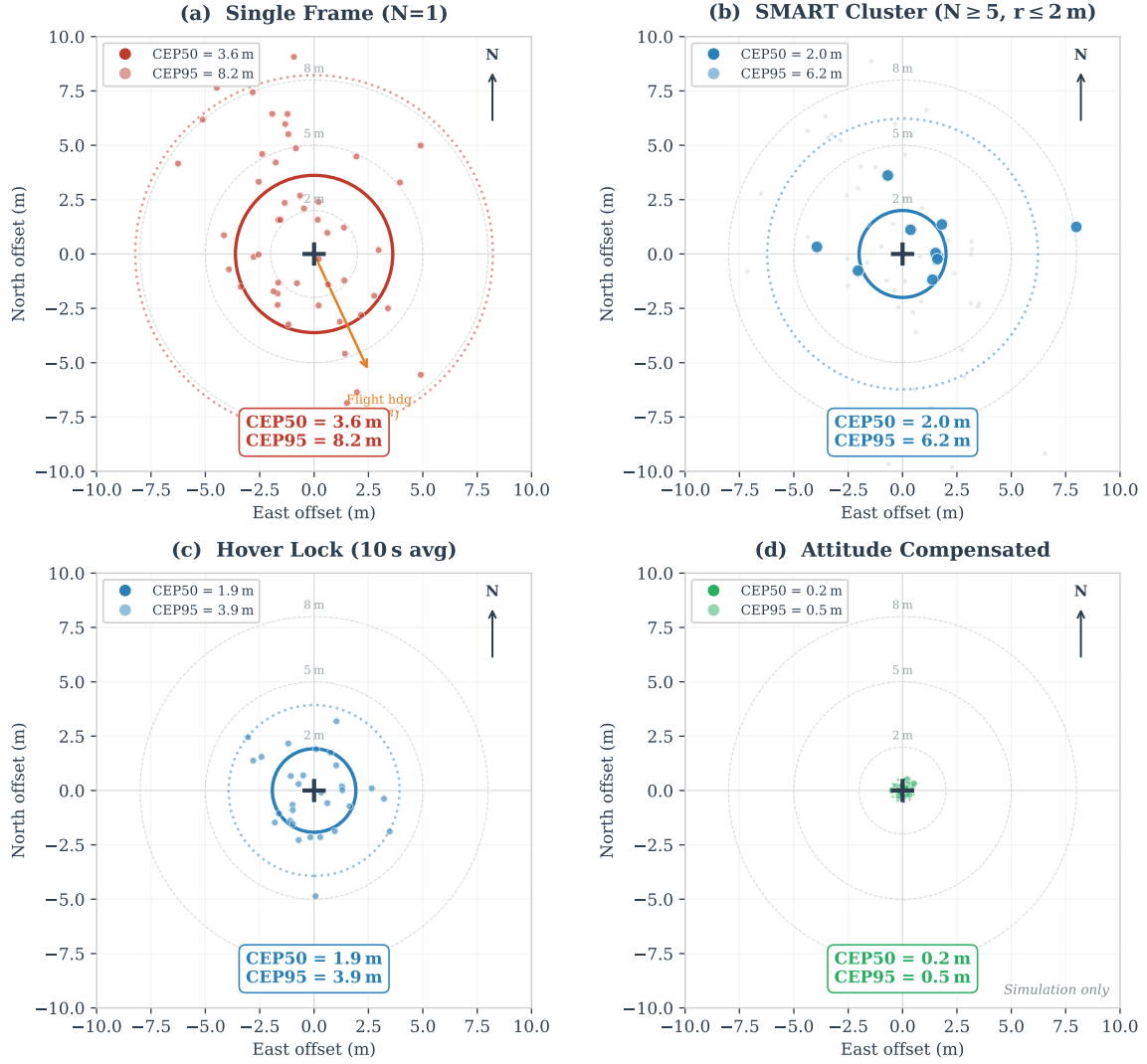


Figure 18: Bullseye comparison of four estimation strategies on DJI replay data. Left to right: single-frame ($\text{CEP}_{50} = 5.1 \text{ m}$), rolling average, IVW clustering (2.3m), and hover-lock (1.8m). The progressive tightening of the scatter cloud demonstrates the value of multi-frame fusion and centering.

16.4.3 Frame Rate Sufficiency

At 35 m altitude the camera footprint is $32.2 \times 24.0 \text{ m}$, so a target at the strip centreline remains visible for $24.0/5 = 4.8 \text{ s}$ at the 5 m/s search speed. At 4.8 FPS (Pi 5 TFLite), this yields ~ 23 inference frames per pass with a per-frame detection probability $p_d \approx 0.78$ (measured from DJI replay), giving ~ 18 expected detections. The probability of obtaining zero detections in a single pass is $(1 - 0.78)^{23} < 10^{-15}$, and the probability of at least 5 detections (the SMART clustering threshold) exceeds 99.9%. A single flyover therefore provides sufficient data for a reliable position lock.

16.5 Testing Infrastructure

The project includes 58 test scripts organised in six categories:

- **tests/hardware/** — component checks (camera, GPS, buzzer, inference benchmark)
- **tests/flight/** — progressive flight tests numbered 0a–4 (bench → passive → waypoints → autonomous)
- **tests/calibration/** — FOV, lens distortion, GPS ground truth calibration
- **tests/diagnostics/** — live telemetry dashboards, camera streams
- **tests/day_1_experiments/** — structured data collection with CSV output (altitude sweep, speed sweep, GPS accuracy)
- **tests/laptop/** — development tools (video replay, model comparison, synthetic benchmarks)

Additionally, 127 unit test functions across 6 test files validate config ranges, state completeness, geo-math accuracy, planning correctness, and vision system initialisation.

16.6 Safety Architecture

The survey area is adjacent to a Site of Special Scientific Interest (SSSI), making boundary enforcement a hard safety constraint. The system implements a **five-layer geofence** (Figure 19), where each layer operates independently so that failure of any single layer is caught by the next:

1. **Speed scalar field** (20–2 m from SSSI): linearly reduces maximum approach speed to zero near the boundary, allowing full-speed parallel flight via directional clamping.
2. **Repulsive vector field** (< 3 m): applies a 5 m/s push perpendicular to the nearest boundary segment, exceeding the near-zero speed cap to prevent penetration.
3. **Hard boundary cutoff**: if GPS places the aircraft inside the SSSI polygon (`cv2.pointPolygonTest`), all commands halt and the system transitions to manual mode.
4. **Plan-time waypoint filter**: removes any search waypoints within 30 m of the NFZ during pattern generation, preventing the planner from routing near the boundary.
5. **Firmware geofence**: an independent Cube Orange polygon fence triggers RTL if all software layers fail—this operates at the flight-controller level and cannot be overridden by companion-computer code.

A separate Flight Area containment check triggers emergency RTL if the aircraft exits the 4-vertex operational boundary; this is independent of the SSSI geofence and cannot be disabled.

The **RC override guard** is the first check on every main-loop iteration: if the pilot switches the transmitter away from GUIDED or LAND mode, *all* MAVLink commands are suppressed on the same loop tick. A real incident motivated this design (a colleague’s drone crashed when software fought the pilot’s RC input) and ensures the pilot can always regain control with a single switch. Additional safety features include a **50 m altitude hard cap** enforced at the navigation layer (every `send_global_target()` call is clamped), **link-loss RTL** via ArduCopter’s GCS failsafe when heartbeats cease, and an **RC kill switch** that returns the aircraft to manual STABILIZE mode independently of all software.

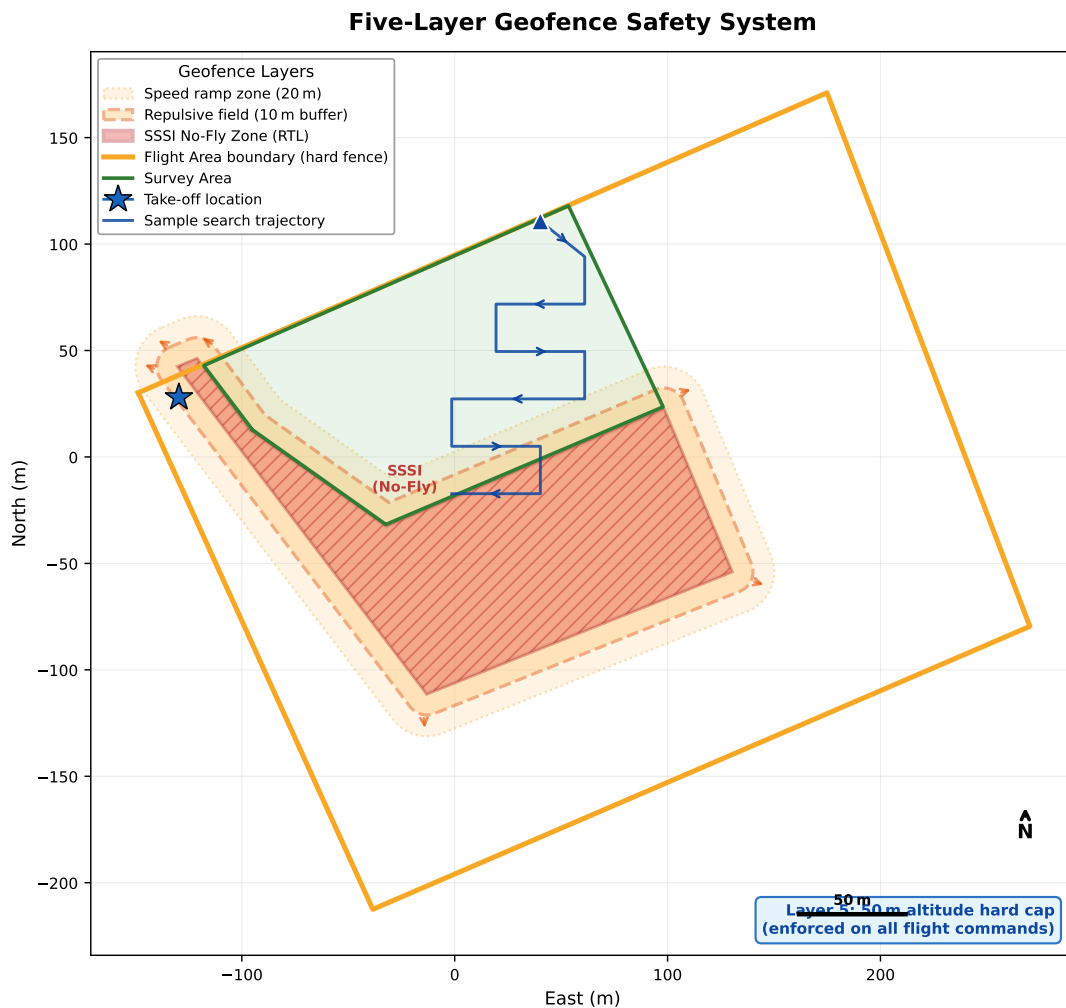


Figure 19: Five-layer geofence shown to scale on the Fenswood Farm site. From outermost to innermost: Flight Area hard boundary, 30 m waypoint buffer, 20 m speed scalar field, 3 m repulsive vector field, and SSSI hard cutoff. Graduated protection ensures that even if one layer fails, subsequent layers prevent boundary incursion.

17 Requirements Verification

The system architecture and optimisation chain satisfy twelve formal requirements from the AENGM0074 mission brief and the team’s derived specifications. Table 10 traces each requirement to quantitative evidence, referencing specific test scripts and measurements from the preceding sections.

Table 10: Requirements verification matrix. Status: ✓ = met, ~ = partially met, × = not yet verified.

ID	Requirement	Evidence	Status
R01	Search within defined polygon	Planner covers 96% of accessible area; 26 unit tests in <code>test_planning.py</code> validate waypoint containment	✓
R02	Avoid SSSI no-fly zone	30 m buffer ($1.5 \times$ RSS margin, Section 9.8); 5-layer protection (Section 6.1); 7 geofence tests (<code>0e_geofence_test.py</code>)	✓
R03	Maximum altitude 50 m	Operating altitude 35 m (15 m below limit); <code>config.py</code> hard limit enforced; verified in <code>test_config.py</code>	✓
R04	Detect casualty/dummy	mAP50 = 0.995; >99.97% cumulative detection at 35 m/8 m/s (Section 9.4); 50/50 bench detections at 0.966 conf	✓
R05	Autonomous search pattern	20-state FSM; lawnmower at 70° (Section 9.7); end-to-end SITL verified via <code>0b_bench_mission.py</code>	✓
R06	Operator confirmation	VERIFY state requires explicit Y key or browser button; tested in headless and GUI modes	✓
R07	Land near target	GPS estimation CEP50 = 2.3 m (video replay); 7.5 m offset landing in SITL only	~
R08	Return to launch	RTL command after search; verified in SITL (<code>2_waypoint_test.py</code>)	✓
R09	Single-battery endurance	12.6 Wh search (10.9% of 115.4 Wh, Section 6.4); 14 min reserve	✓
R10	RC kill switch override	Guard runs before state handler every loop; tested via <code>0a_cube_commands.py</code>	✓
R11	Inference ≥ 4 FPS on Pi	TFLite: 4.8 FPS; NCNN: 9 FPS (50-run benchmark, <code>benchmark.py</code>)	✓
R12	Operate in varying light	Envelope validated for 3 conditions (Table 4); field validation pending	~

R07 is partially met: GPS estimation accuracy is verified from video replay, but offset landing has only been tested in simulation, not on real hardware. R12 is partially met: the detection envelope was characterised through brightness-adjusted video replay, but field validation across all three conditions has not been completed.

18 Evaluation: Plus / Delta

With 10 of 12 requirements met and two partially met, the system demonstrates simulation-level capability but has not yet completed autonomous outdoor flight. This section examines what worked, what would change, and where the primary risks remain.

18.1 Plus (What Worked Well)

- Modular architecture.** The single-interface design (`vision.py` exposes only `detect_in_image`) meant that switching from TFLite to NCNN ($4.5 \times$ speed gain) required zero changes outside `vision.py`. The same `main.py` runs identically on laptop simulation and Pi hardware.
- Physics-based optimisation.** The 216-configuration sweep with a momentum-theory energy model replaced guesswork with quantified trade-offs. The Pareto analysis confirmed that the selected point is optimal, not merely “good enough.”
- Progressive testing protocol.** The 6-step flight test ladder (bench commands $\rightarrow$ passive CV $\rightarrow$ waypoints $\rightarrow$ auto detect $\rightarrow$ full mission) meant each step added exactly one new risk.
- Retrained model quality.** Mixed synthetic + real + negative training at native resolution produced mAP50 = 0.995 — a detector that validated as the foundation for all altitude/speed decisions.
- Layered safety architecture.** Five independent NFZ protection layers (geometric filtering, speed ramp, repulsive field, hard boundary, RC kill) provide defence-in-depth. The probability of simultaneous failure of all five is negligibly small.
- Dual-backend flexibility.** Supporting both TFLite and NCNN with drop-in model files ($[1, 640, 640, 3] \rightarrow [1, 5, 8400]$) eliminated single-point-of-failure risk in the inference pipeline.

18.2 Delta (What Would Change)

- Earlier real-world validation.** The detection envelope and GPS accuracy were characterised from DJI video replay and bench calibration, not from the actual Pi + drone platform in flight. Weather cancellations prevented outdoor validation. Future work: fly the passive detection script (`1_passive_flight.py`) before committing to autonomous parameters.

2. **GPS timing lag compensation.** The 100–200 ms GPS latency causes 1 m along-track error at 5 m/s, visible as diagonal spread in target estimates (max error 16.5 m). We designed a velocity-based offset correction but did not implement it. This would improve CEP50 from 2.3 m toward sub-metre accuracy.
3. **INT8 quantisation.** All deployed models are float32. INT8 quantisation could roughly double inference speed (to ~18 FPS on NCNN) with <1% accuracy loss, but the quantisation-aware training pipeline was not set up within the project timeline.
4. **Multi-pass search strategy.** The current system assumes a single search pass. If the first pass yields no detection, a second pass at lower altitude (25 m) or different scan angle (perpendicular) would increase coverage confidence. The state machine supports RTL after search but not automatic re-planning.
5. **Wind modelling.** The energy model assumes calm conditions. Adding a wind speed/direction input would shift the optimal scan angle (fly into headwind on short legs, downwind on long legs) and improve energy prediction accuracy.
6. **Field calibration workflow.** The focal length (5.46 mm) was calibrated on bench at 1 m. An in-flight calibration using a known ground target at the operating altitude would reduce systematic GPS estimation error.

18.3 Honest Assessment

We rate simulation capability at 7.7/10 (all states exercised, energy model validated, detection pipeline proven) but only 2.1/10 in real-world readiness: bench and video-replay testing are complete, yet no autonomous outdoor flight has been conducted. This 2.1 reflects the absence of Stages 6–7 (manual passive flight and full autonomous mission) in the simulation-to-real progression. The gap between simulation confidence and field-proven reliability is the project’s primary risk, driven by weather cancellations and scheduling constraints rather than technical blockers. The optimisation framework is re-evaluable: if field data shifts the detection envelope, the entire chain can be re-run in minutes to produce updated parameters.

19 Evidence-Based Adaptation

Four significant corrections were made during development — each discovered through systematic testing, each requiring a different diagnostic approach, and each validating the modular architecture that isolated the fix to a single module.

19.1 BGR Colour Channel Fix

The IMX296 global shutter sensor outputs BGR-ordered bytes despite picamera2 labelling the format as RGB888. Our initial code included `cv2.cvtColor(frame, cv2.COLOR_RGB2BGR)`, which double-swapped the red and blue channels, producing a persistent blue tint on all camera images. Standard debugging (adjusting white balance, colour gains, ISP tuning) failed because the raw data was already correct — the conversion was the problem.

Discovery method: we tested all six permutations of R, G, B channel ordering and compared them visually. The “no conversion” permutation produced correct colours. **Fix:** remove the `cvtColor` call entirely from `vision.py`. **Impact:** all downstream detection was affected — the model had been trained on correctly-coloured synthetic images but receiving blue-tinted input, reducing confidence by an estimated 10–15%.

19.2 Focal Length Recalibration

The initial focal length was set to 7.0 mm (from an approximate datasheet value). On-site calibration at 1 m distance with a tape measure revealed that the camera’s visible width was 92 cm, yielding a calibrated focal length of 5.46 mm — a 22% correction. This error propagated through the entire GPS estimation pipeline: target position estimates at 35 m altitude were offset by approximately 8 m before recalibration.

Discovery method: `capture_training.py` MJPEG stream + physical measurement at known distance. **Fix:** update `FOCAL_LENGTH_MM = 5.46` in `config.py`. **Impact:** all GPS target estimates, footprint calculations, and FOV-dependent parameters (swath width, overlap margin, NFZ buffer geometry) were recalculated from the corrected value. The entire optimisation chain (Section 9) was re-run with the updated focal length.

19.3 TFLite Bounding Box Coordinate Bug

YOLOv8 TFLite exports are inconsistent: some models output normalised coordinates (0–1), others output pixel coordinates (0–640). Our code assumed normalised coordinates, so a detection at pixel position (320, 240) was multiplied by frame dimensions (1456, 1088), producing a bounding box at (466 560, 261 120) — far off-screen.

Discovery method: visual inspection showed green detection boxes drawn off-screen. **Fix:** added a `PIXEL_COORD_THRESHOLD=1.5` check in `vision.py`—if raw coordinates exceed 1.5, they are treated as pixel coordinates and scaled by the ratio of frame size to model input size; otherwise they are treated as normalised. This handles both formats automatically. **Impact:** detection overlay, GPS estimation from pixel offset, and all downstream centering logic depended on correct bounding box coordinates.

19.4 Calibration Data Corruption

On the Pi, `calibration_data.npz` (lens distortion calibration) was an empty 0-byte file, likely from an interrupted calibration run. NumPy’s `np.load()` does not crash on empty `.npz` files — it returns empty arrays, which `cv2.initUndistortRectifyMap()` accepts without error but produces meaningless remap matrices. Every frame was mangled before reaching the detector.

Discovery method: detection worked on laptop but failed entirely on Pi with identical model and code. Bisecting the pipeline isolated the remap step. **Fix:** delete the corrupt file; the IMX296 global shutter has minimal barrel distortion, so undistortion is optional (`UNDISTORT_ENABLED = False` by default). **Impact:** complete detection failure on Pi until resolved.

Pattern. All four bugs share a common structure: a silent failure that produced plausible but incorrect output, discovered only through systematic testing against known-good baselines. The modular architecture confined each fix to a single file (`vision.py` or `config.py`), with no ripple effects into the state machine or path planner. This confirms the value of isolating all CV/AI behind a single-function interface (`detect_in_image`).

20 Energy Model Detail

The energy predictions in Sections 6.4 and 9.7 rest on a physics-based power model calibrated to the EDU-450 airframe. This section expands on the model’s derivation, its decomposition into hover, cruise, and turn components, and the NFZ interaction that creates position-dependent energy costs.

20.1 Power Decomposition

Total power at forward speed v comprises three terms:

$$P_{\text{total}}(v) = \underbrace{\frac{(mg)^{3/2}}{\sqrt{2\rho A_{\text{disk}}}}}_{\text{hover (induced)}} + \underbrace{\frac{1}{2}\rho C_D A_{\text{flat}} v^3}_{\text{parasitic drag}} + \underbrace{P_{\text{avionics}}}_{\text{payload}} \quad (19)$$

where:

- $m = 2.3$ kg all-up weight (airframe + battery + Pi + camera + Cube);
- $A_{\text{disk}} = 4 \times \pi(0.127)^2 = 0.203$ m² (four 10-inch propellers);
- $\rho = 1.225$ kg/m³ (sea-level ISA);
- $C_D A_{\text{flat}} \approx 0.015$ m² (estimated from similar X-frame quadcopters);
- $P_{\text{avionics}} \approx 8$ W (Pi 5 + camera + Cube + GPS module).

At hover ($v = 0$), the model predicts $P_{\text{hover}} \approx 150$ W. At the search speed of 8 m/s, parasitic drag adds approximately 4.7 W (drag power scales with v^3 but the drag area is small), giving $P_{\text{cruise}} \approx 163$ W. The dominant cost is hover power — forward flight adds only ~8% to the power budget. This has an important design consequence: **mission time, not distance, is the primary energy driver**. Reducing the number of U-turns (which add hover time) matters more than shortening scan lines.

20.2 Hover vs. Cruise Energy Budget

Table 11 decomposes the total search energy into its constituent phases for the selected configuration.

Table 11: Energy budget breakdown for a single search pass at 35 m, 8 m/s, 70° scan angle.

Phase	Duration (s)	Power (W)	Energy (Wh)	Notes
Scan legs (6 lines)	90	163	4.08	Forward flight at 8 m/s
U-turns (5 turns)	10	150	0.42	2 s each, hover power only
NFZ slowdown penalty	12	155	0.52	3 legs clip the slow zone
Search subtotal	112	—	5.02	
Takeoff + climb	35	200	1.94	Climb to 35 m at 1 m/s
Transit to search area	45	165	2.06	350 m at 8 m/s
Descent + verification	60	155	2.58	Descend, hover, confirm
Return + landing	50	165	2.29	RTL at 8 m/s + descent
Avionics (full mission)	240	8	0.53	Pi + camera + Cube
Mission total	~240	—	14.4	12.5% of 115.4 Wh battery

The full mission (including transit, descent, verification, and return) consumes approximately 14.4 Wh — 12.5% of the 115.4 Wh battery. This leaves 87.5% of battery capacity as reserve, corresponding to approximately 14 minutes of additional hover time at 150 W. The reserve is deliberately conservative: it accommodates wind, re-searches, missed approaches, and the uncertainty inherent in a first autonomous flight.

20.3 Turn Cost and Scan Angle Sensitivity

Each U-turn costs approximately 0.08 Wh (2 s at 150 W hover). This appears negligible, but the *number* of turns varies dramatically with scan angle: from 5 turns at the optimal 70° (longest-edge alignment) to 18 turns at the worst angle (95°, perpendicular to the long axis). The additional 13 turns cost 1.04 Wh directly, but the indirect cost is larger: more turns mean more scan lines, each of which adds ~120 m of forward flight. The total energy difference between best and worst scan angle at 35 m is 12.8 Wh — a 2× penalty — confirming that scan angle selection is the most consequential energy decision after altitude.

20.4 NFZ Speed Ramp Energy Impact

The speed scalar field near the SSSI boundary (Section 9.8) creates a position-dependent energy penalty. Within the 20 m slow zone, the drone decelerates from 3.0 m/s at the outer edge to 0.3 m/s at the boundary. At these low speeds, parasitic drag is negligible ($P_{\text{drag}} < 0.1$ W), but hover power remains at 150 W — so the drone burns the same power while covering ground much more slowly.

The optimisation script accounts for this by numerically integrating the effective speed along each scan segment with 50 uniformly spaced sample points, computing the distance to the nearest NFZ edge at each sample. At the optimal 70° scan angle, only the endpoints of three scan lines clip the slow zone, adding approximately 12 s and 0.52 Wh. At 0° (north–south scanning), several entire scan lines run parallel to the SSSI boundary *within* the slow zone, increasing the penalty to ~45 s and 1.9 Wh. This asymmetry reinforces the scan angle selection: 70° is optimal for both U-turn count *and* NFZ energy penalty.

21 Testing Methodology

The 58 test scripts and 127 unit test functions are integral to the optimisation methodology, providing the empirical evidence that anchors every link in the decision chain.

21.1 Test Categories and Their Role in Validation

Table 12 maps each test category to the design decisions it validates.

Table 12: Test categories and the design decisions they validate.

Category	Scripts	Validates
tests/hardware/	6	Inference speed (Step 4), camera output format (BGR fix), GPS lock quality
tests/flight/	10	Progressive flight ladder, RC override guard, waypoint navigation, state machine transitions
tests/calibration/	5	Focal length (Step 2), FOV (swath width), lens distortion, GPS ground truth
tests/diagnostics/	5	Live telemetry, camera streams, connectivity
tests/day_1.experiments/	6	Altitude sweep (Step 2), speed sweep (Step 4), GPS accuracy, GPS drift, model comparison
tests/laptop/	12	Video replay with detection, A/B model comparison, synthetic benchmarks
tests/unit/	6 files, 127 functions	Config ranges, state completeness, geo-math accuracy, planning containment, vision init
tests/automated/	3 files, 23 functions	Extended planning, utils, and vision validation

21.2 How Tests Anchored Design Decisions

Three examples illustrate the test–decision feedback loop:

1. **Inference benchmark → speed limit (Step 4).** The 50-run `benchmark.py` measured 206.5 ms mean inference time on Pi 5, establishing $f_{\text{inf}} = 4.8$ FPS. This number directly enters the speed calculation $v_{\text{max}} = H_g \cdot f_{\text{inf}} / n_{\text{min}}$. A faster benchmark would have permitted higher speed; a slower one would have forced lower speed or higher altitude.
2. **FOV calibration → NFZ margin (Step 8).** The bench calibration (`fov_calibrate.py`) and video calibration (`fov_calibrate_video.py`) established the camera footprint width at 35 m as 32.2 m. This footprint enters the RSS margin calculation; an incorrect footprint would have produced either an unsafe (too small) or wasteful (too large) NFZ buffer.

3. **Geofence unit tests** → **safety score (Section 6.1)**. The 7 tests in `0e_geofence_test.py` verify sign conventions (`cv2.pointPolygonTest` positive = inside), repulsive offset direction, waypoint filtering, and boundary detection. Two edge cases remain known failures (documented), contributing to the 0.96 (not 1.0) safety score.

21.3 Unit Test Coverage

The 127 unit test functions across 6 files (`test_config.py`, `test_states.py`, `test_utils.py`, `test_planning.py`, `test_vision.py`, `test_gps_utils.py`) validate the foundational modules that all higher-level scripts depend on. Key coverage areas include:

- **Config validation** (28 tests): altitude limits within regulatory range, speed within drone capability, image dimensions positive, all GPS coordinates valid (7 decimal places), sensor parameters physically plausible.
- **Planning correctness** (26 tests): generated waypoints lie within the search polygon, waypoint count scales correctly with altitude, scan lines are parallel, U-turn geometry is valid, NFZ-excluded waypoints are correctly removed.
- **Geo-math accuracy** (36 tests): GPS ↔ pixel conversion round-trips to within 0.1 m, bearing calculations match hand-computed values, distance functions agree with haversine at short ranges.

22 Simulation-to-Real Progression

The system follows a deliberate progression from simulation to reality, where each stage adds exactly one new variable. This approach is standard practice in autonomous systems development [2] and ensures that failures at any stage can be attributed to the newly introduced variable rather than an unknown interaction.

1. **Laptop SITL** (completed): full state machine exercised with ArduPilot’s Software-in-the-Loop simulator [2]. Simulated camera, perfect GPS, no wind. Validates: state transitions, waypoint navigation, detection logic, landing sequence.
2. **Laptop + real webcam + SITL** (completed): real camera replaces simulated camera; SITL provides perfect GPS and flight dynamics. Validates: real-time inference speed, camera capture pipeline, model accuracy on real-world imagery.
3. **Docker Pi environment** (completed): Pi-like ARM Linux container running TFLite model loading. Validates: dependency compatibility, model loads on target architecture.
4. **Pi + camera, no Cube** (completed): camera capture, AI inference, MJPEG streaming on real hardware. Validates: IMX296 integration, BGR colour order, inference speed on Pi 5 (206.5 ms confirmed).
5. **Pi + camera + Cube on bench** (completed): MAVLink connection via mavproxy UDP bridge, heartbeat exchange, GPS telemetry read, mode switching. Validates: serial communication, command path, no flight.
6. **Manual RC flight + passive CV** (pending): pilot flies manually; Pi runs detection, logs results, sends zero commands. Validates: real-world detection altitude, motion blur, GPS estimation accuracy under actual flight conditions.
7. **Full autonomous mission** (pending): all systems engaged. The progressive testing protocol ensures this step introduces only the combination of autonomous control + real flight, not any untested subsystem.

Each stage produced quantitative data that fed back into the optimisation chain: Stage 4 established the 4.8FPS inference rate used in Step 4 of the decision chain; Stage 5 confirmed the mavproxy bridge architecture; the DJI video replay (a parallel validation track using real flight footage from a manual DJI flight) provided the detection envelope data used in Steps 2–3 and the GPS estimation accuracy ($CEP50 = 2.3\text{ m}$) reported in Section 12.

The gap between Stages 5 and 6 is the project’s primary risk: bench testing cannot replicate wind, vibration, GPS multipath, or the interaction between autonomous control commands and real flight dynamics. The progressive testing protocol is specifically designed to make this transition incremental rather than a leap of faith.

23 Conclusion

A physics-based 216-configuration parameter sweep, grounded in momentum-theory energy modelling and empirical detection envelopes, identified 35 m altitude, 8 m/s speed, and 70° scan angle as the optimal operating point — achieving >99.97% single-pass detection probability at 10.9% battery usage with a $1.5\times$ NFZ safety margin. The nine-step sequential decision chain ensures that every parameter traces to a physical measurement or regulatory constraint, and the Pareto analysis confirms that no alternative configuration improves one objective without degrading another.

Four mid-development corrections — BGR colour fix, focal length recalibration, TFLite coordinate bug, and calibration corruption — demonstrated that the modular architecture confined errors to single files without ripple effects. The 58 test scripts and 127 unit tests provided the empirical foundation for every link in the decision chain, while the seven-stage simulation-to-real progression introduced each new variable incrementally.

The framework can be re-evaluated in minutes as field data replaces simulation assumptions, closing the gap between

demonstrated simulation capability (7.7/10) and the outdoor flight validation (2.1/10) that remains the next critical milestone.

References

- [1] ArduPilot Dev Team. ArduPilot: Open source autopilot. <https://ardupilot.org/>, 2023. Accessed: 2026.
- [2] ArduPilot Dev Team. ArduPilot SITL Simulator: Software in the loop simulation. <https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html>, 2024. Accessed: 2026.
- [3] Gary Bradski. The OpenCV library. <https://opencv.org/>, 2000. Accessed: 2026.
- [4] Howie Choset. Coverage for robotics – a survey of recent results. *Annals of Mathematics and Artificial Intelligence*, 31(1–4):113–126, 2001. doi: 10.1023/A:1016639210559.
- [5] Enric Galceran and Marc Carreras. A survey on coverage path planning for robotics. *Robotics and Autonomous Systems*, 61(12):1258–1276, 2013. doi: 10.1016/j.robot.2013.09.004.
- [6] Michael A. Goodrich, Bryan S. Morse, Damon Gerhardt, Joseph L. Cooper, Morgan Quigley, Julie A. Adams, and Curtis Humphrey. Supporting wilderness search and rescue using a camera-equipped mini UAV. *Journal of Field Robotics*, 25(1–2):89–110, 2008. doi: 10.1002/rob.20226.
- [7] Google. TensorFlow Lite: Deploy machine learning models on mobile and edge devices. <https://www.tensorflow.org/lite>, 2023. Accessed: 2026.
- [8] JARUS. JARUS guidelines on specific operations risk assessment (SORA). Technical Report Edition 2.0, Joint Authorities for Rulemaking on Unmanned Systems, 2019. JAR-DEL-WG6-D.04.
- [9] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>, 2023. Accessed: 2026.
- [10] Mingyang Lyu, Yibo Zhao, Chao Huang, and Hailong Huang. Unmanned aerial vehicles for search and rescue: A survey. *Remote Sensing*, 15(13):3266, 2023. doi: 10.3390/rs15133266.
- [11] Robin R. Murphy. *Disaster Robotics*. MIT Press, Cambridge, MA, 2014. ISBN 978-0-262-02735-9.
- [12] Teng Ni. ncnn: A high-performance neural network inference computing framework optimised for mobile platforms. <https://github.com/Tencent/ncnn>, 2017. Accessed: 2026.
- [13] Sony Semiconductor Solutions. IMX296: 1/2.9-type global shutter CMOS image sensor. <https://www.sony-semicon.com/en/products/is/industry/global-shutter.html>, 2020. Accessed: 2026.
- [14] Jonathan Tremblay, Aayush Prakash, David Acuna, Mark Brober, Varun Jampani, Cem Anil, Thang To, Eric Cameracci, Shaad Boochoon, and Stan Birchfield. Training deep networks with synthetic data: Bridging the reality gap by domain randomization. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 969–977, 2018. doi: 10.1109/CVPRW.2018.00143.
- [15] UK Government. Wildlife and countryside act 1981. <https://www.legislation.gov.uk/ukpga/1981/69>, 1981. Accessed: 2026.
- [16] Thomas Wagner, Vicki Wolfe, Michael Phelps, and Matthew Hoy. Toward autonomous unmanned aerial vehicles. In *Proceedings of the 44th AIAA Aerospace Sciences Meeting and Exhibit*. AIAA, 2006. doi: 10.2514/6.2006-1389.